

# Laporan Tugas Besar

## Teori Bahasa Formal dan Automata

### Milestone 2 - Syntax Analysis



**Disusun oleh kelompok :**  
Kerjakan J - 5

|                              |          |
|------------------------------|----------|
| Mahatma Brahmana             | 13524015 |
| Benedict Darrel Setiawan     | 13524057 |
| Raymond Jonathan Dwi Putra J | 13524059 |
| Reynard Anderson Wijaya      | 13524111 |

**Program Studi Teknik Informatika**  
**Sekolah Teknik Elektro dan Informatika**  
**Institut Teknologi Bandung**  
**2026**

## Daftar Isi

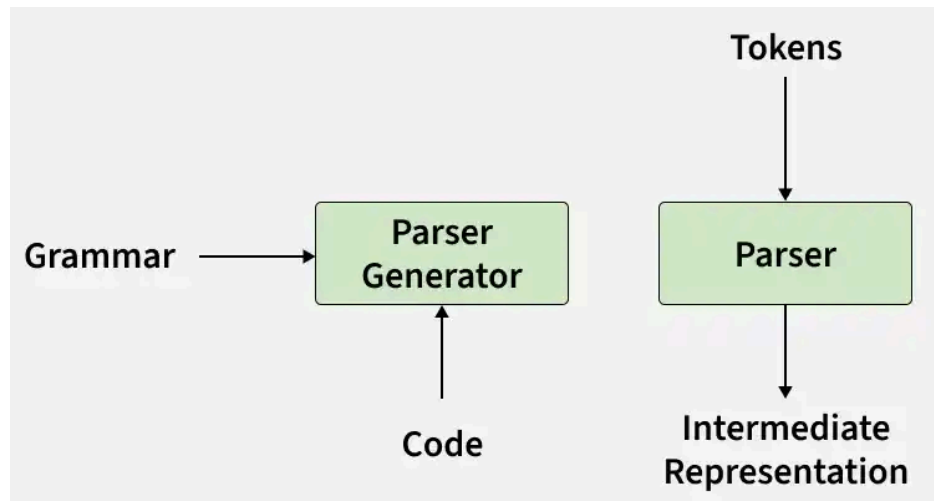
|                                                                        |           |
|------------------------------------------------------------------------|-----------|
| <b>Daftar Isi.....</b>                                                 | <b>2</b>  |
| <b>BAB 1 Landasan Teori.....</b>                                       | <b>4</b>  |
| 1.1 Parser.....                                                        | 4         |
| 1.2 Parse Tree.....                                                    | 4         |
| <b>BAB 2 Perancangan Program.....</b>                                  | <b>5</b>  |
| 2.1 Gambaran Umum dan Teknologi yang Digunakan.....                    | 5         |
| 2.2 Struktur Program dan Fungsi/Kelas Utama.....                       | 5         |
| 2.2.1 Struktur Direktori Utama.....                                    | 6         |
| 2.2.2 Fungsi dan Kelas Utama Program.....                              | 6         |
| 2.3 Alur Kerja.....                                                    | 8         |
| 2.4 Daftar Node dan Grammar yang Didukung.....                         | 9         |
| 2.4.1 Daftar Node.....                                                 | 9         |
| 2.4.2 Grammar yang Didukung.....                                       | 14        |
| <b>BAB 3 Implementasi.....</b>                                         | <b>14</b> |
| 3.1 Implementasi Program Pemanggilan Parser dan Menampilkan Hasil..... | 14        |
| 3.2 Implementasi Helper method.....                                    | 14        |
| 3.3 Implementasi Node Program( ).....                                  | 15        |
| 3.4 Implementasi Node Program-header.....                              | 16        |
| 3.5 Implementasi Node Declaration-part.....                            | 16        |
| 3.6 Implementasi Node Const-declaration.....                           | 17        |
| 3.7 Implementasi Node Constant.....                                    | 18        |
| 3.8 Implementasi Node Type-declaration.....                            | 19        |
| 3.9 Implementasi Node Type.....                                        | 20        |
| 3.10 Implementasi Node Array-type.....                                 | 21        |
| 3.11 Implementasi Node Range.....                                      | 21        |
| 3.12 Implementasi Node Enumerated.....                                 | 22        |
| 3.13 Implementasi Node Record-type.....                                | 23        |
| 3.14 Implementasi Node Field-list.....                                 | 23        |
| 3.15 Implementasi Node Field-part.....                                 | 24        |
| 3.16 Implementasi Node Var-declaration.....                            | 24        |
| 3.17 Implementasi Node Identifier-list.....                            | 25        |
| 3.18 Implementasi Node Subprogram-declaration.....                     | 25        |
| 3.19 Implementasi Node Procedure-declaration.....                      | 26        |
| 3.20 Implementasi Node Function-declaration.....                       | 27        |
| 3.21 Implementasi Node Block.....                                      | 27        |
| 3.22 Implementasi Node Formal-parameter-list.....                      | 28        |
| 3.23 Implementasi Node Parameter-group.....                            | 28        |
| 3.24 Implementasi Node Compound-statement.....                         | 29        |
| 3.25 Implementasi Node Statement-list.....                             | 30        |

|                                                      |           |
|------------------------------------------------------|-----------|
| 3.26 Implementasi Node Statement.....                | 30        |
| 3.27 Implementasi Node Assignment-statement.....     | 31        |
| 3.28 Implementasi Node If-statement.....             | 31        |
| 3.29 Implementasi Node Case-statement.....           | 32        |
| 3.30 Implementasi Node Case-block.....               | 33        |
| 3.31 Implementasi Node While-statement.....          | 33        |
| 3.32 Implementasi Node Repeat-statement.....         | 34        |
| 3.33 Implementasi Node For-statement.....            | 34        |
| 3.34 Implementasi Node Procedure/function-call.....  | 35        |
| 3.35 Implementasi Node Parameter-list.....           | 36        |
| 3.36 Implementasi Node Variable.....                 | 36        |
| 3.37 Implementasi Node Component-variable.....       | 37        |
| 3.38 Implementasi Node Index-list.....               | 38        |
| 3.39 Implementasi Node Expression.....               | 38        |
| 3.40 Implementasi Node Simple-expression.....        | 39        |
| 3.41 Implementasi Node Term.....                     | 39        |
| 3.42 Implementasi Node Factor.....                   | 40        |
| 3.43 Implementasi Node Relational-operator.....      | 41        |
| 3.44 Implementasi Node Additive-operator.....        | 42        |
| 3.45 Implementasi Node Multiplicative-operator.....  | 42        |
| 3.46 Implementasi Program Pencetakan Parse Tree..... | 43        |
| <b>BAB 4 Pengujian.....</b>                          | <b>45</b> |
| 4.1 Pengujian Masukkan dan Keluaran Program.....     | 45        |
| <b>BAB 5</b>                                         |           |
| <b>Kesimpulan dan Saran.....</b>                     | <b>50</b> |
| 5.1 Kesimpulan.....                                  | 50        |
| 5.2 Saran.....                                       | 50        |
| <b>Lampiran.....</b>                                 | <b>51</b> |

## BAB 1 Landasan Teori

### 1.1 Parser

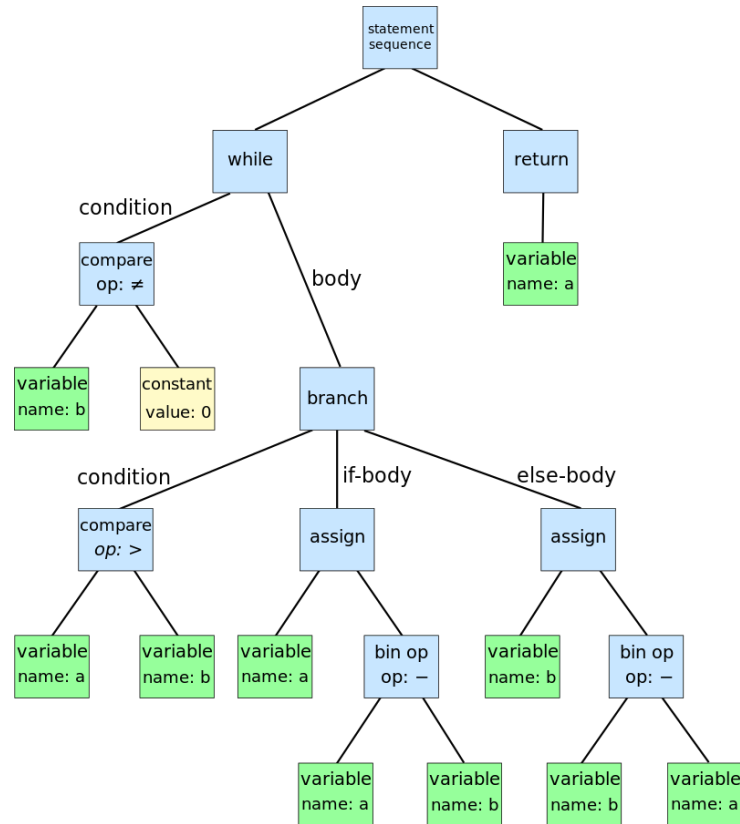
Langkah kedua dari proses *compiling* merupakan *parsing*. *Parsing*, atau dapat disebut juga sebagai *syntax analysis*, merupakan proses untuk menganalisis sekuens token yang dihasilkan oleh *lexical analyser* untuk menentukan struktur *grammatical* dari program dengan menyusunnya ke dalam sebuah *Immediate Representation* (IR). *Parsing* juga memastikan bahwa sekuens token mengikuti aturan *syntax* dari bahasa pemrograman yang di-compile untuk analisis lanjut dan eksekusi.



Gambar 1.1 Alur Parser

### 1.2 Parse Tree

*Immediate Representation* (IR) yang paling sering menjadi hasil dari sebuah parser merupakan *parse tree*. *Parse Tree*, atau dapat disebut juga sebagai *Abstract Syntax Tree*, merupakan representasi hierarkis sebuah *string* yang mengikuti suatu *grammar*. Setiap node di dalam *parse tree* dapat dibagi menjadi dua tipe yaitu *terminal* dan *non-terminal*. Node *terminal* merupakan ujung dari *parse tree* sehingga tidak memiliki child node. Node *terminal* dalam sebuah *compiler* bahasa pemrograman biasanya berupa sebuah simbol atau token yang didapatkan dari hasil *lexical analyser*. Node *non-terminal* merupakan node yang menjadi interior dari *parse tree* dan biasanya merepresentasikan suatu struktur di sebuah *grammar* (Contoh: ekspresi, deklarasi, for-statement, etc). Fungsi utama dari *parse tree* di sebuah *compiler* merupakan memudahkan visualisasi dan analisis struktur *syntax* dari sebuah program, yang akan berperan dalam pendeteksian *error* serta optimisasi.



Gambar 1.2 Contoh Parse Tree

### 1.3 Recursive Descent Parser

*Recursive Descent Parser* merupakan sebuah *top-down parser* yang menggunakan serangkaian fungsi rekursif untuk menguraikan input berdasarkan aturan *grammar*. *Parser* ini memanfaatkan fungsi secara rekursif ke kanan untuk membangun *parse tree* sesuai dengan *grammar* yang ditentukan. Algoritma *parsing* ini akan mencoba melakukan *matching* untuk setiap rekursi yang dilakukan hingga *matching* sesuai atau gagal. Jika sebuah fungsi gagal melakukan *matching*, maka *parsing* dianggap gagal dan *parse tree* tidak akan tersusun. Berikut adalah algoritma dari *Recursive Descent Parser*:

1. Mulai di sebuah *non-terminal*
2. Memilih salah satu aturan *grammar* untuk melakukan *matching terminal*
3. Jika *matching* gagal maka lanjut ke alternatif
4. Ulangi 2-3 hingga terjadi *matching* atau hingga tidak terdapat lagi alternatif
5. Jika terjadi *matching* dan aturan mengandung sebuah *non-terminal*, ulangi 2-4 dengan *non-terminal* tersebut
6. Jika tidak terjadi *matching* maka *parsing* gagal dan algoritma berhenti.

## BAB 2 Perancangan Program

### 2.1 Gambaran Umum dan Teknologi yang Digunakan

Pada milestone kedua ini, program yang dikembangkan ini adalah lanjutan dari program *lexical analyzer* pada milestone pertama. Pada milestone pertama, program berfokus untuk memecah *source code* menjadi token-token. Pada milestone kedua ini, program dilanjutkan dengan menambahkan bagian *syntax analyzer* atau parser. Parser ini berfungsi untuk memeriksa apakah urutan token yang dihasilkan oleh *lexer* sudah sesuai dengan aturan grammar penulisan bahasa Arion.

Program ini umumnya bekerja dengan menerima masukan berupa file input (*source code* Arion). File tersebut akan diproses terlebih dahulu oleh *lexer* untuk menghasilkan daftar token. Setelah itu, daftar token tersebut akan diberikan kepada parser untuk diperiksa strukturnya/urutannya. Jika susunan token sudah sesuai dengan aturan *grammar* yang digunakan, parser akan membentuk *parse tree* yang menunjukkan susunan program. Hasil *parse tree* tersebut kemudian ditampilkan di terminal dan disimpan ke dalam file *output*. Jika ada token yang tidak sesuai dengan aturan *grammar*, program akan menampilkan pesan error pada bagian yang bermasalah.

Program ini dibuat menggunakan bahasa C++. Bahasa pemrograman ini dipakai karena program pada milestone sebelumnya juga dibuat menggunakan C++, sehingga pengembangan parser dapat langsung melanjutkan struktur program yang sudah ada. Selain itu, C++ menyediakan struktur data seperti *vector*, *string*, dan *class*, yang dapat membantu dalam penyimpanan token, pembuatan *node parse tree*, dan lain-lainnya.

Program ini juga menggunakan Makefile untuk proses kompilasi supaya proses build dapat dilakukan dengan lebih mudah. Dengan adanya Makefile ini, file-file program dapat dikompilasi melalui satu perintah. Hal ini membuat proses jalannya program menjadi lebih mudah dan praktis.

### 2.2 Struktur Program dan Fungsi/Kelas Utama

Untuk memastikan kemudahan pengembangan, pembacaan kode, serta kelancaran integrasi pada tahap kompilasi selanjutnya (seperti Syntax dan Semantic Analysis), program Lexical Analyzer ini dirancang dengan arsitektur yang modular. Program dipisahkan ke dalam beberapa bagian berdasarkan tugasnya masing-masing. Secara umum, program terdiri dari program utama, modul *lexer*, dan modul parser. Pemisahan ini dilakukan supaya setiap bagian program memiliki tanggung jawab yang jelas dan lebih mudah diperiksa ketika terjadi kesalahan.

Pada milestone ini, bagian *lexer* dari milestone pertama masih digunakan sebagai tahap awal untuk mengubah *source code* Arion menjadi daftar token. Setelah token terbentuk, daftar token tersebut

akan diproses oleh parser untuk diperiksa berdasarkan aturan grammar bahasa Arion. Hasil akhir dari proses parser adalah parse tree yang menunjukkan susunan program.

### 2.2.1 Struktur Direktori Utama

Struktur utama program pada milestone ini adalah sebagai berikut.

```
src/  
├── main.cpp  
├── lexer/  
│   ├── lexer.hpp  
│   ├── lexer.cpp  
│   ├── token.hpp  
│   └── token.cpp  
└── parser/  
    ├── parser.hpp  
    └── parser.cpp
```

Penjelasan dari masing-masing bagian direktori adalah sebagai berikut:

1. **src/** : Merupakan direktori utama yang menyimpan seluruh source code program.
2. **src/main.cpp** : Merupakan file utama yang mengatur jalannya program. File ini menghubungkan proses pembacaan input, tokenisasi oleh lexer, parsing oleh parser, dan pencetakan hasil parse tree.
3. **src/lexer/** : Merupakan folder yang berisi bagian lexical analyzer dari milestone pertama. Bagian ini digunakan untuk membaca source code Arion dan mengubahnya menjadi daftar token.
4. **src/parser/** : Merupakan folder yang berisi bagian syntax analyzer atau parser. Bagian ini digunakan untuk memeriksa susunan token dan membentuk parse tree.
5. **doc/ dan test/** : Merupakan folder yang digunakan untuk menyimpan laporan.
6. **test/** : Merupakan folder yang digunakan untuk menyimpan file input dan output pengujian.
7. **Makefile** : Digunakan untuk membantu proses kompilasi program agar dapat dijalankan dengan lebih mudah melalui satu perintah.

### 2.2.2 Fungsi dan Kelas Utama Program

Dalam program ini, terdapat beberapa fungsi dan kelas utama yang digunakan untuk menjalankan proses dari source code sampai menjadi parse tree.

#### 1. Modul Token (**token.hpp** dan **token.cpp**) :

Modul ini bertanggung jawab penuh atas definisi dan representasi struktur data token.

- **enum ListToken:** Digunakan untuk mendaftarkan seluruh (52 jenis) token yang valid di dalam bahasa pemrograman Arion, yang dikelompokkan mulai dari literal, operator aritmatika, operator logika, hingga *keyword* kontrol alur program.
- **Kelas Token:** Berfungsi sebagai wadah untuk menyimpan entitas token yang berhasil diidentifikasi. Kelas ini menyimpan tipe token (**ListToken**) dan nilai leksikal aktualnya dalam bentuk `std::string`. Terdapat pula metode `tokenTypeToString()` yang mereturn representasi teks dari suatu token untuk mempermudah pencetakan keluaran.

## 2. Modul Lexer (**lexer.hpp** dan **lexer.cpp**) :

Modul ini merupakan mesin inti yang menjalankan logika *Deterministic Finite Automata* (DFA).

- **enum State:** Mendefinisikan secara komprehensif seluruh transisi *state* yang mungkin terjadi selama pembacaan aliran karakter. Ini mencakup *state* dasar seperti pembacaan angka (**INT\_STATE**), string (**STRING\_ONGOING\_STATE**), hingga rincian transisi karakter demi karakter pembentuk *keyword* (misalnya transisi menuju **PROGRAM\_STATE**).
- **Fungsi tokenize():** Merupakan fungsi utama pada modul ini yang menerima parameter berupa nama berkas (*file name*). Fungsi ini bertugas mengelola siklus DFA dengan membaca karakter satu per satu, menggeser transisi *state* sesuai karakter masukan, lalu mengembalikan hasilnya dalam bentuk senarai (`vector<Token>`).

## 3. Modul Parser (**parser.hpp** dan **parser.cpp**) :

Modul ini merupakan bagian utama yang ditambahkan pada milestone kedua. Modul parser bertugas menjalankan proses syntax analysis, yaitu memeriksa apakah susunan token dari lexer sudah sesuai dengan aturan grammar bahasa Arion. Jika susunan token sesuai, parser akan membentuk parse tree. Jika tidak sesuai, parser akan menampilkan pesan error.

- **Struktur TreeParser :** Struktur ini digunakan untuk merepresentasikan node pada parse tree. Setiap node menyimpan nama bagian grammar atau token, serta daftar anak dari node tersebut. Dengan adanya struktur ini, hasil parsing dapat disusun dalam bentuk pohon yang menunjukkan hubungan antarbagian program.
- **Kelas Parser :** Kelas ini merupakan bagian utama yang menjalankan proses parsing. Kelas Parser menyimpan daftar token yang diterima dari lexer dan posisi token yang sedang diperiksa. Dengan informasi tersebut, parser dapat membaca token secara berurutan dari awal sampai akhir.

- **Fungsi bantuan parser :** Di dalam kelas Parser, terdapat beberapa fungsi bantuan untuk melihat token saat ini, mengecek token berikutnya, dan mencocokkan token dengan aturan grammar. Fungsi-fungsi ini membantu proses parsing agar lebih mudah diatur dan tidak perlu menulis pengecekan token secara berulang-ulang.
- Pada proses parsing, kelas Parser menggunakan metode Recursive Descent. Dengan metode ini, setiap bagian grammar dibuat sebagai fungsi tersendiri, misalnya fungsi untuk membaca program utama, deklarasi, statement, ekspresi, term, dan factor. Cara ini membuat alur parser lebih mudah diikuti karena setiap fungsi hanya menangani satu bagian grammar tertentu.

#### 4. Program Utama (main.cpp)

Berkas ini bertindak sebagai titik awal eksekusi program. Pada milestone kedua, main.cpp tidak hanya menjalankan proses lexer, tetapi juga menghubungkannya dengan proses parser. Dengan begitu, file ini berperan sebagai pengatur alur utama dari pembacaan input sampai penyimpanan hasil parse tree.

- Manajemen I/O: Program menerima masukan melalui Command Line Arguments (argc dan argv). Pengguna perlu memberikan path file input yang berisi source code Arion dan path file output yang akan digunakan untuk menyimpan hasil parse tree.
- Alur Pemanggilan: Program akan memanggil fungsi tokenize() dengan menggunakan path file input. Fungsi ini menghasilkan daftar token dalam bentuk vector<Token>. Setelah itu, daftar token tersebut diberikan kepada objek Parser untuk diproses. Parser kemudian menjalankan proses parsing dan membentuk parse tree. Jika proses parsing berhasil, hasil parse tree akan dicetak ke terminal dan disimpan ke file output. Jika terdapat kesalahan pada susunan token, program akan menampilkan pesan error agar bagian yang bermasalah dapat diketahui.

## 2.3 Alur Kerja

Alur kerja program pada milestone kedua ini dimulai dari pembacaan file input yang berisi source code Arion. File input tersebut diberikan oleh pengguna saat program dijalankan. Selain file input, pengguna juga memberikan file output yang akan digunakan untuk menyimpan hasil akhir program, yaitu parse tree.

Pertama, program akan menjalankan lexer. Lexer ini akan membaca isi source code Arion dari file input, lalu mengubahnya menjadi daftar token. Setiap token menyimpan jenis token dan nilai yang dibaca dari source code. Hasil dari proses lexer ini kemudian diberikan kepada parser untuk diproses lebih lanjut.

Setelah parser menerima daftar token yang sudah dibentuk, parser akan mulai membaca token-token tersebut secara berurutan. Parser akan memeriksa apakah urutan token sudah sesuai dengan aturan grammar bahasa Arion. Jika susunan token sudah sesuai, parser akan membentuk parse tree yang menunjukkan struktur program.

Pada proses parsing, parser akan memanggil fungsi-fungsi yang sesuai dengan bagian grammar yang sedang diperiksa. Misalnya, ketika parser membaca bagian awal program, parser akan memproses bagian program utama dan program header. Ketika parser masuk ke bagian deklarasi, parser akan memeriksa deklarasi konstanta, tipe, variabel, prosedur, atau fungsi jika bagian tersebut ada. Setelah itu, parser akan memeriksa bagian statement yang berada di dalam blok begin dan end.

Jika parser menemukan token yang tidak sesuai dengan aturan grammar, proses parsing akan dihentikan dan program akan menampilkan pesan error. Pesan error ini digunakan untuk membantu pengguna mengetahui bagian mana yang bermasalah. Dengan begitu, kesalahan pada source code dapat lebih mudah diperiksa dan diperbaiki.

Hasil akhir dari program adalah parse tree yang ditampilkan ke terminal dan disimpan ke dalam file output. Tampilan di terminal berguna agar hasil parsing dapat langsung dilihat saat program dijalankan. Sementara itu, penyimpanan ke file output berguna agar hasil parse tree dapat digunakan kembali sebagai bukti pengujian dan dokumentasi hasil program.

## 2.4 Daftar Node dan Grammar yang Didukung

Pada program ini, parse tree dibentuk dari node-node yang mengikuti aturan grammar bahasa Arion. Setiap node mewakili bagian tertentu dari program, sedangkan grammar digunakan parser untuk mengecek urutan token dari lexer. Daftar node utama dan aturan grammar yang didukung oleh program dijelaskan pada bagian berikut.

### 2.4.1 Daftar Node

Pada program ini, parse tree dibentuk dari node-node yang mengikuti aturan grammar bahasa Arion.

Daftar node yang digunakan dalam program ini dapat dilihat pada tabel berikut :

| Nama Node | Deskripsi                                                  | Aturan Produksi                                                                       | Contoh                                          |
|-----------|------------------------------------------------------------|---------------------------------------------------------------------------------------|-------------------------------------------------|
| <program> | Node root yang merepresentasikan keseluruhan program Arion | program $\rightarrow$ program-header + declaration-part + compound-statement + period | Seluruh struktur program dari awal hingga akhir |

|                     |                                                                                                                |                                                                                             |                                                                               |
|---------------------|----------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <program-header>    | Bagian kepala program yang berisi nama program                                                                 | programsy + ident + semicolon                                                               | program Hello;                                                                |
| <declaration-part>  | Bagian deklarasi yang dapat berisi const, type, var, procedure, function                                       | (const-declaration)* + (type-declaration)* + (var-declaration)* + (subprogram-declaration)* | Semua deklarasi sebelum begin                                                 |
| <const-declaration> | Deklarasi konstanta                                                                                            | constsy + (ident + eql + constant + semicolon)+                                             | const<br>INT == 100;<br>REAL == 10.0;<br>CHAR == 'a';<br>STRING ==<br>'abcd'; |
| <constant>          | Konstanta dapat berupa ident, intcon, realcon, charcon, atau string                                            | charcon   string   [(plus   minus)? + (ident   intcon   realcon)]                           | a'<br>'abcd'<br>10<br>+10.0<br>-a                                             |
| <type-declaration>  | Deklarasi tipe data baru                                                                                       | typesy + (ident + eql + type + semicolon)+                                                  | type Range ==<br>1..10;<br>type Month ==<br>(Jan, Feb, Mar,<br>Apr);          |
| <var-declaration>   | Deklarasi variabel                                                                                             | varsy + (identifier-list + colon + type + semicolon)+                                       | var a, b:<br>integer;                                                         |
| <identifier-list>   | Daftar identifier yang dipisahkan koma                                                                         | ident (comma + ident)*                                                                      | a, b, c                                                                       |
| <type>              | Tipe data yang terdefinisi atau didefinisikan pengguna. Dapat berupa ident, array-type, range, atau enumerated | ident   array-type   range   enumerated   record-type                                       | integer,<br>array[1..10] of<br>integer                                        |
| <array-type>        | Definisi tipe array                                                                                            | arraysy + lbrack + (range   ident) + rbrack + ofsy + type                                   | array[1..10] of<br>integer<br><br>array[char] of<br>integer                   |

|                          |                                                     |                                                                                              |                                                                                                                                   |
|--------------------------|-----------------------------------------------------|----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <range>                  | Rentang nilai untuk array atau subrange             | constant + period + period + constant                                                        | 1..10, 'a'..'z'                                                                                                                   |
| <enumerated>             | Nilai tertentu yang ada di dalam list               | lparent + ident + (comma + ident)* + rparent                                                 | (Jan, Feb, Mar, Apr)                                                                                                              |
| <record-type>            | Tipe data template, sama dengan class dalam OOP     | recordsy + field-list + endsy                                                                | type<br>record-name =<br>record<br>field-1:<br>field-type1;<br>field-2:<br>field-type2;<br>...<br>field-n:<br>field-typen;<br>end |
| <field-list>             | Penjabaran atribut/field dari record                | field-part + (semicolon + field-part)*                                                       | ..field-1..<br>..field-2..                                                                                                        |
| <field-part>             | Singular atribut/field dari record                  | identifier-list + colon + type                                                               | field-1:<br>field-type1;                                                                                                          |
| <subprogram-declaration> | Deklarasi prosedur atau fungsi                      | procedure-declaration   function-declaration                                                 | procedure<br>print(x:<br>integer);                                                                                                |
| <procedure-declaration>  | Deklarasi prosedur                                  | proceduresy + ident + (formal-parameter-list)? + semicolon + block + semicolon               | Prosedur dengan parameter opsional                                                                                                |
| <function-declaration>   | Deklarasi fungsi                                    | functionsy + ident + (formal-parameter-list)? + colon + ident + semicolon+ block + semicolon | Fungsi yang mengembalikan nilai                                                                                                   |
| <block>                  | Bagian kode yang mengandung deklarasi dan statement | declaration-part + compound-statement                                                        | var<br>...<br>begin<br>...<br>end                                                                                                 |
| <formal-parameter-list>  | Daftar parameter formal                             | lparent + parameter-group + (semicolon + parameter-group)* +                                 | (x, y: integer;<br>z: real)                                                                                                       |

|                        |                                                                 |                                                                                                                                        |                                                                              |
|------------------------|-----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
|                        |                                                                 | parent                                                                                                                                 |                                                                              |
| <parameter-group>      | Daftar parameter disertai dengan tipe data parameter            | identifier-list + colon + (ident   array-type)                                                                                         | x, y: integer                                                                |
| <compound-statement>   | Blok statement yang diawali begin dan diakhiri end              | beginsy + statement-list + endsy                                                                                                       | begin ...<br>end                                                             |
| <statement-list>       | Daftar statement yang dipisahkan semicolon                      | statement (semicolon + statement)*                                                                                                     | Urutan statement dalam blok                                                  |
| <statement>            | Deskripsi aksi yang harus dilakukan program komputer            | (assignment-statement   if-statement   case-statement   while-statement   repeat-statement   for-statement   procedure/function-call)? | x := 5;<br>if x > 0 then y := 1<br>else y := 0<br>while x < 10 do x := x + 1 |
| <variable>             | Variable dapat berbentuk ident, elemen array, atau field record | ident + (component-variable)*                                                                                                          | someVar<br>someArray[1]<br>someField                                         |
| <component-variable>   | Variable dalam bentuk element array atau field record           | (lbrack + index-list + rbrack)   (period + ident)                                                                                      | someArray[1]<br>someRecord.someField<br>someArray[1][2]                      |
| <index-list>           | Penunjuk index dalam array                                      | ( intcon   charcon   ident ) + ( comma + index-list )*                                                                                 | [1, 2, 3]<br>['a', 'b']<br>[true, false]                                     |
| <assignment-statement> | Statement penugasan nilai ke variabel                           | variable + becomes + expression                                                                                                        | x := 5, y := x + 10                                                          |
| <if-statement>         | Statement kondisional                                           | ifsy + expression + thensy + statement + (elsy + statement)?                                                                           | if x > 0 then y := 1 else y := 0                                             |
| <case-statement>       | Statement kondisional dengan banyak kondisional                 | casesy + expression + ofsy + case-block + endsy                                                                                        | case i of<br>0: x := 0;<br>1 : x := 1;<br>end                                |
| <case-block>           | Deklarasi case dan statement yang harus dilakukan               | constant + (comma + constant)* + colon + statement + (semicolon +                                                                      | 0: x := 0;<br>1, 2: x := 1                                                   |

|                           |                                               |                                                                                                                                        |                                      |
|---------------------------|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|
|                           |                                               | case-block?)*                                                                                                                          |                                      |
| <while-statement>         | Perulangan dengan kondisi di awal             | whilesy + expression + dosy + statement                                                                                                | while x < 10 do<br>x := x + 1        |
| <repeat-statement>        | Perulangan dengan kondisi di akhir            | repeatsy + statement-list + untilsy + expression                                                                                       | repeat<br>x := x - 1<br>until x == 0 |
| <for-statement>           | Perulangan dengan counter                     | forsy + ident + becomes + expression + ( tosy   downtosy) + expression + dosy + statement                                              | for i := 1 to 10 do ...              |
| <procedure/function-call> | Pemanggilan prosedur atau fungsi              | ident + (lparent + parameter-list? + rparent)?                                                                                         | writeln('Hello'), print(x, y)        |
| <parameter-list>          | Daftar parameter aktual saat pemanggilan      | expression (comma + expression)*                                                                                                       | Result = ', b, 100                   |
| <expression>              | Ekspresi yang menghasilkan nilai              | simple-expression (relational-operator + simple-expression)?                                                                           | x + y, a > b, 5 * (x + 1)            |
| <simple-expression>       | Ekspresi tanpa operator relasional            | (plus   minus)? term (additive-operator + term)*                                                                                       | x + y - z, 5 * 2                     |
| <term>                    | Bagian ekspresi dengan prioritas lebih tinggi | factor (multiplicative-operator + factor)*                                                                                             | x * y, a div b                       |
| <factor>                  | Unit terkecil dalam ekspresi                  | ident   intcon   realcon   charcon   string   (lparent + expression + rparent)   (notsy + factor)   procedure/function-call   variable | x, 5, 'a', (x+y), not a              |
| <relational-operator>     | Operator perbandingan                         | eq   neq   gtr   geq   lss   leq                                                                                                       | x == 5, y > 10                       |
| <additive-operator>       | Operator penjumlahan/pengurangan              | plus   minus   orsy                                                                                                                    | x + y, a or b                        |
| <multiplicative-operator> | Operator perkalian/pembagian                  | times   rdiv   idiv   imod   andsy                                                                                                     | x * y, a div b, p and q              |

### 2.4.2 Grammar yang Didukung

Pada program ini, parse tree dibentuk berdasarkan aturan grammar bahasa Arion. Node pada parse tree umumnya berasal dari bagian non-terminal grammar, yaitu bagian yang ditulis dengan tanda <...>, seperti <program>, <statement>, <expression>, dan <factor>. Sementara itu, token seperti programsy, ident, semicolon, beginsy, dan intcon merupakan terminal yang berasal dari hasil lexer.

Notasi yang digunakan pada penulisan grammar adalah sebagai berikut :

|       |                                            |
|-------|--------------------------------------------|
| A   B | berarti memilih salah satu antara A atau B |
| A*    | berarti A dapat muncul nol kali atau lebih |
| A+    | berarti A muncul satu kali atau lebih      |
| A?    | berarti A bersifat opsional                |

Grammar yang didukung oleh program adalah sebagai berikut :

```

<program> → <program-header> <declaration-part>
<compound-statement> period

<program-header> → programsy ident semicolon

<declaration-part> → <const-declaration>* <type-declaration>*
<var-declaration>* <subprogram-declaration>*

<const-declaration> → constsy (ident eql <constant> semicolon)+

<constant> → charcon
            | stringcon
            | (plus | minus)? (ident | intcon | realcon)

<type-declaration> → typesy (ident eql <type> semicolon)+

<type> → ident
        | <array-type>
        | <range>
        | <enumerated>
        | <record-type>

<array-type> → arraysy lbrack (ident | <range>) rbrack ofsy <type>

<range> → <expression> period period <expression>

<enumerated> → lparent ident (comma ident)* rparent

<record-type> → recordsy <field-list> endsy

<field-list> → <field-part> (semicolon <field-part>)* semicolon?

<field-part> → <identifier-list> colon <type>

```

```

<var-declaration> → varsy (<identifier-list> colon <type>
semicolon)+

<identifier-list> → ident (comma ident)*

<subprogram-declaration> → <procedure-declaration>
                           | <function-declaration>

<procedure-declaration> → proceduresy ident
<formal-parameter-list>? semicolon <block> semicolon

<function-declaration> → functionsy ident <formal-parameter-list>?
colon ident semicolon <block> semicolon

<block> → <declaration-part> <compound-statement>

<formal-parameter-list> → lparent <parameter-group> (semicolon
<parameter-group>)* rparent

<parameter-group> → <identifier-list> colon (ident | <array-type>)

<compound-statement> → beginsy <statement-list> endsy

<statement-list> → <statement> (semicolon <statement>)*

<statement> → <assignment-statement>
              | <if-statement>
              | <case-statement>
              | <while-statement>
              | <repeat-statement>
              | <for-statement>
              | <procedure-function-call>
              | <compound-statement>
              | ε

<assignment-statement> → <variable> becomes <expression>

<if-statement> → ifsy <expression> thensy <statement> (elsesy
<statement>)?

<case-statement> → casesy <expression> ofsy <case-block> endsy

<case-block> → <case-label-list> colon <statement> (semicolon
<case-block> | semicolon)?

<case-label-list> → <constant> (comma <constant>)*

<while-statement> → whilesy <expression> dosy <statement>

<repeat-statement> → repeatsy <statement-list> untilsy <expression>

```

```

<for-statement> → forsy ident becomes <expression> (tosy |
downtosy) <expression> dosy <statement>

<procedure-function-call> → ident (lparent <parameter-list>?
rparent)?

<parameter-list> → <expression> (comma <expression>)*

<variable> → ident <component-variable>*

<component-variable> → (lbrack <index-list> rbrack) | (period
ident)

<index-list> → (intcon | charcon | ident) (comma (intcon | charcon
| ident))*

<expression> → <simple-expression> (<relational-operator>
<simple-expression>)?

<simple-expression> → (plus | minus)? <term> (<additive-operator>
<term>)*

<term> → <factor> (<multiplicative-operator> <factor>)*

<factor> → <variable>
          | ident lparent <parameter-list>? rparent
          | intcon
          | realcon
          | charcon
          | stringcon
          | lparent <expression> rparent
          | notsy <factor>

<relational-operator> → eql | neq | gtr | geq | lss | leq

<additive-operator> → plus | minus | orsy

<multiplicative-operator> → times | rdiv | idiv | imod | andsy

```

## BAB 3 Implementasi

### 3.1 Implementasi Program Pemanggilan Parser dan Menampilkan Hasil

Program parser diimplementasikan menggunakan algoritma Recursive Descent Parser. Parser menerima hasil keluaran dari lexer berupa list token, kemudian melakukan pengecekan struktur sintaks

berdasarkan grammar bahasa Arion dan menghasilkan parse tree sebagai output. Sebelum proses parsing dilakukan, program membuat objek Parser melalui konstruktor. Konstruktor tersebut menyimpan list token yang sudah dibersihkan dari token comment serta menginisialisasi posisi pembacaan token pada indeks awal.

```
// Pada fungsi main.cpp
vector<Token> tokens = tokenize(inputFile);
Parser parser(tokens);
TreeParser* parseTree = parser.parse();
parser.printParseTree(parseTree, outputFile);

// Pada fungsi parser.cpp
Parser::Parser(const std::vector<Token>& tokens) :
tokens(Parser::cleanCommentTokens(tokens)), pos(0) {}

std::vector<Token> Parser::cleanCommentTokens(const
std::vector<Token>& tokens) const {
    std::vector<Token> cleanedTokens;
    for (const Token& token : tokens) {
        if (token.type != comment) {
            cleanedTokens.push_back(token);
        }
    }
    return cleanedTokens;
}
```

### 3.2 Implementasi Helper method

Dalam implementasi parser, terdapat beberapa helper method yang digunakan untuk membantu proses parsing agar lebih terstruktur dan mudah digunakan kembali pada berbagai node grammar. Helper method ini berfungsi untuk melakukan pembacaan token, pengecekan token, pencocokan token, hingga konversi token menjadi representasi string pada parse tree maupun pesan error.. Dengan adanya helper method ini, implementasi parser menjadi lebih sederhana dan tidak perlu melakukan pengecekan token secara manual secara berulang.

| Method         | Fungsi                                                    |
|----------------|-----------------------------------------------------------|
| currentToken() | Mengambil token pada posisi saat ini                      |
| check()        | Mengecek apakah token saat ini memiliki tipe tertentu     |
| checkNext()    | Mengecek token berikutnya tanpa memindahkan posisi parser |
| peekToken()    | Melihat token pada offset tertentu                        |
| match()        | Mencocokkan token dengan grammar yang diharapkan          |

|                                   |                                          |
|-----------------------------------|------------------------------------------|
| <code>cleanCommentTokens()</code> | Menghapus token komentar sebelum parsing |
|-----------------------------------|------------------------------------------|

### 3.3 Implementasi Node Program()

Secara umum, proses parsing dimulai dari fungsi `parse()` yang kemudian memanggil fungsi `Program()` sebagai root dari grammar.

```
TreeParser* Parser::parse() {
    return Parser::Program();
}
```

Fungsi `Program()` menjadi titik awal pembentuk parse tree. Fungsi ini membentuk node `<program>` dan memanggil bagian-bagian utama program Arion, yaitu `ProgramHeader`, `DeclarationPart`, `Compound Statement`, dan node `period`.

```
TreeParser* Parser::Program() {
    try {
        TreeParser* node = new TreeParser("<program>");
        node->addChild(Parser::ProgramHeader());
        node->addChild(Parser::DeclarationPart());
        node->addChild(Parser::CompoundStatement());
        node->addChild(Parser::match(period));
        if ((size_t)pos < tokens.size()) {
            throw std::runtime_error("unexpected token after
program end: " + tokenToErrorString(tokens[pos]));
        }
        return node;
    } catch (const std::runtime_error& e) {
        if (startsWith(e.what(), "Syntax error")) {
            throw;
        }
        throw std::runtime_error(std::string("Syntax error: ") +
e.what());
    }
}
```

Penggunaan exception pada method ini bertujuan untuk menangani kesalahan sintaks (syntax error) yang terjadi selama proses parsing berlangsung. Ketika parser membaca token dan menemukan token yang tidak sesuai dengan aturan grammar, fungsi pada node yang sedang diproses akan melempar (`throw`) `std::runtime_error`. Error tersebut kemudian diteruskan ke node di atasnya hingga mencapai fungsi `Program()` sebagai root parser.

Melalui mekanisme ini, parser dapat menghentikan proses parsing ketika terjadi kesalahan dan memberikan pesan error yang lebih informatif terkait lokasi atau bagian grammar yang menyebabkan

error. Kesalahan yang ditangani pada tahap ini termasuk ke dalam compile-time error, khususnya syntax error yang terjadi akibat susunan token tidak sesuai dengan grammar bahasa Arion.

### 3.4 Implementasi Node Program-header

Fungsi ProgramHeader() merupakan pengecekan pada bagian kepala program yang berisikan nama program dari pembuatan kode bahasa pemrograman Arion. Fungsi akan mengecek apakah terdapat token programsy, ident, dan semicolon yang digunakan untuk membaca "Program <nama program> ;"

```
TreeParser* Parser::ProgramHeader() {
    try{
        TreeParser* node = new TreeParser("<program-header>");
        node->addChild(Parser::match(programsy));
        node->addChild(Parser::match(ident));
        node->addChild(Parser::match(semicolon));
        return node;
    }
    catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
ProgramHeader: ") + e.what());
    }
}
```

Penggunaan exception pada method ini bertujuan untuk memberitahu bahwa terjadi kesalahan sintaks pada bagian node Program Header.

### 3.5 Implementasi Node Declaration-part

Fungsi DeclarationPart() merupakan fungsi pengecekan pada bagian deklarasi program Arion yang dapat berisikan const, type, var, procedure, dan juga functionnya. Fungsi ini akan mengecek apakah terdapat node const-declaration, node type-declaration, node var-declaration, dan node subprogram-declaration. Fungsi ini akan memanggil setiap fungsi dari node-node tersebut dan mengembalikan node. Pada declaration-part, urutan deklarasi telah ditentukan, yaitu const, type, var, dan subprogram. Apabila urutan yang dideklarasikan tidakurut, maka akan mengembalikan std::runtime\_error dan juga informasi terkait permasalahan yang ada.

```
TreeParser* Parser::DeclarationPart() {
    TreeParser* node = new TreeParser("<declaration-part>");
    try{
        bool TypeDecl = false, VarDecl = false, SubprogramDecl =
false;
        while(Parser::check(constsy) || Parser::check(typesy) ||
Parser::check(varsy) || Parser::check(proceduresy) ||
Parser::check(functionsy)) {
```

```

        if(Parser::check(constsy) && !TypeDecl && !VarDecl &&
!SubprogramDecl) {
            node->addChild(Parser::ConstDeclaration());
        }
        else if(Parser::check(typesy) && !VarDecl &&
!SubprogramDecl) {
            node->addChild(Parser::TypeDeclaration());
            TypeDecl = true;
        }
        else if(Parser::check(varsy) && !SubprogramDecl) {
            node->addChild(Parser::VarDeclaration());
            VarDecl = true;
            TypeDecl = true;
        }
        else if(Parser::check(proceduresy) ||
Parser::check(functionsy)) {
            node->addChild(Parser::SubprogramDeclaration());
            SubprogramDecl = true;
            VarDecl = true;
            TypeDecl = true;
        }
        else {
            throw std::runtime_error("invalid declaration order: use
const, type, var, then procedure/function");
        }
    }
    return node;
}
catch (const std::runtime_error& e) {
    throw std::runtime_error(std::string("Syntax error in
DeclarationPart: ") + e.what());
}
}

```

### 3.6 Implementasi Node Const-declaration

Fungsi ConstDeclaration() merupakan fungsi pengecekan pada bagian deklarasi konstanta dalam bahasa Arion. Fungsi ini digunakan untuk memvalidasi deklarasi konstanta yang diawali dengan keyword const dan diikuti oleh satu atau lebih pasangan identifier dan nilai konstanta. Fungsi akan memeriksa keberadaan token constsy, kemudian melakukan parsing terhadap identifier, token eql sebagai operator deklarasi konstanta, nilai konstanta, serta token semicolon sebagai penutup deklarasi. Proses parsing dilakukan secara berulang selama masih ditemukan token identifier berikutnya. Jika struktur deklarasi konstanta tidak sesuai dengan grammar yang telah ditentukan, maka fungsi akan melempar std::runtime\_error beserta informasi kesalahan sintaks yang terjadi.

```

TreeParser* Parser::ConstDeclaration() {

```

```

TreeParser* node = new TreeParser("<const-declaration>");
try{
    if(Parser::check(constsy)){
        node->addChild(Parser::match(constsy));
        do {
            node->addChild(Parser::match(ident));
            node->addChild(Parser::match(eql));
            node->addChild(Parser::Constant());
            node->addChild(Parser::match(semicol));
        } while (Parser::check(ident));
    } else {
        throw std::runtime_error("expected const keyword");
    }
    return node;
}
catch (const std::runtime_error& e) {
    throw std::runtime_error(std::string("Syntax error in
ConstDeclaration: ") + e.what());
}
}

```

### 3.7 Implementasi Node Constant

Fungsi Constant() merupakan fungsi pengecekan terhadap nilai konstanta pada bahasa Arion. Fungsi ini akan memvalidasi apakah token yang diterima berupa karakter (charcon), string (stringcon), integer (intcon), real (realcon), maupun identifer yang dapat diawali operator unary plus atau minus. Fungsi akan membangun node <constant> dan menambahkan child sesuai token yang dikenali. Jika token yang diterima tidak sesuai dengan definisi grammar konstanta, maka fungsi akan menghasilkan std::runtime\_error yang menjelaskan kesalahan sintaks pada bagian constant.

```

TreeParser* Parser::Constant() {
    TreeParser* node = new TreeParser("<constant>");
    try {
        if (Parser::check(charcon)) {
            node->addChild(Parser::match(charcon));
        } else if (Parser::check(stringcon)) {
            node->addChild(Parser::match(stringcon));
        } else {
            // (plus | minus)?
            if (Parser::check(plus)) {
                node->addChild(Parser::match(plus));
            } else if (Parser::check(minus)) {
                node->addChild(Parser::match(minus));
            }
            // (ident | intcon | realcon)
            if (Parser::check(ident)) {
                node->addChild(Parser::match(ident));
            }
        }
    }
}

```

```

        } else if (Parser::check(intcon)) {
            node->addChild(Parser::match(intcon));
        } else if (Parser::check(realcon)) {
            node->addChild(Parser::match(realcon));
        } else {
            throw std::runtime_error("expected identifier,
integer literal, or real literal in constant");
        }
    }
    return node;
}
catch (const std::runtime_error& e) {
    throw std::runtime_error(std::string("Syntax error in
Constant: ") + e.what());
}
}

```

### 3.8 Implementasi Node Type-declaration

Fungsi TypeDeclaration() merupakan fungsi pengecekan pada deklarasi tipe data baru dalam bahasa Arion. Fungsi ini memvalidasi deklarasi yang diawali dengan keyword type, kemudian diikuti identifier, token eql, definisi tipe data, dan diakhiri token semicolon. Fungsi melakukan parsing secara berulang selama masih ditemukan identifier berikutnya sehingga memungkinkan deklarasi beberapa tipe data dalam satu blok type. Apabila struktur deklarasi tipe tidak sesuai grammar, maka fungsi akan melempar `std::runtime_error` beserta informasi kesalahan sintaks.

```

TreeParser* Parser::TypeDeclaration() {
    TreeParser* node = new TreeParser("<type-declaration>");
    try {
        node->addChild(Parser::match(typesy));
        do {
            node->addChild(Parser::match(ident));
            node->addChild(Parser::match(eql));
            node->addChild(Parser::Type());
            node->addChild(Parser::match(semicolon));
        } while (Parser::check(ident));
        return node;
    }
    catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
TypeDeclaration: ") + e.what());
    }
}

```

### 3.9 Implementasi Node Type

Fungsi Type() merupakan fungsi pengecekan terhadap definisi tipe data dalam bahasa Arion. Fungsi ini akan menentukan jenis tipe data yang sedang diproses, yaitu identifier biasa, array, record, enumerated type, maupun range type. Pemeriksaan dilakukan berdasarkan token awal yang ditemukan, seperti arraysy untuk array, recordsy untuk record, dan lparent untuk enumerated type. Jika token berupa identifier dan diikuti operator period (.), maka parser akan menganggapnya sebagai range type. Fungsi akan memanggil parser node yang sesuai dan mengembalikan node <type>. Jika struktur tipe tidak valid, maka fungsi akan menghasilkan std::runtime\_error.

```

TreeParser* Parser::Type() {
    TreeParser* node = new TreeParser("<type>");
    try {
        if (Parser::check(arraysy)) {
            node->addChild(Parser::ArrayType());
        } else if (Parser::check(recordsy)) {
            node->addChild(Parser::RecordType());
        } else if (Parser::check(lparent)) {
            // enumerated diawali '('
            node->addChild(Parser::Enumerated());
        } else if (Parser::check(ident)) {
            if (Parser::checkNext(period)) {
                node->addChild(Parser::Range());
            } else {
                node->addChild(Parser::match(ident));
            }
        } else {
            // intcon/realcon/charcon/plus/minus
            node->addChild(Parser::Range());
        }
        return node;
    }
    catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in Type: ") + e.what());
    }
}

```

### 3.10 Implementasi Node Array-type

Fungsi ArrayType() merupakan fungsi pengecekan terhadap deklarasi array dalam bahasa Arion. Fungsi ini memvalidasi struktur array yang terdiri dari keyword array, index range atau identifier di dalam bracket, keyword of, dan tipe elemen array. Fungsi akan memanggil parser Range() apabila indeks array berupa rentang nilai, atau melakukan matching identifier jika indeks menggunakan tipe tertentu. Setelah

itu fungsi akan memproses tipe elemen array melalui parser `Type()`. Jika terdapat kesalahan sintaks pada deklarasi array, maka fungsi akan melempar `std::runtime_error`.

```
TreeParser* Parser::ArrayType() {
    TreeParser* node = new TreeParser("<array-type>");
    try {
        node->addChild(Parser::match(arraysy));
        node->addChild(Parser::match(lbrack));
        if (Parser::check(ident) && Parser::checkNext(rbrack)) {
            node->addChild(Parser::match(ident));
        } else {
            node->addChild(Parser::Range());
        }
        node->addChild(Parser::match(rbrack));
        node->addChild(Parser::match(ofsy));
        node->addChild(Parser::Type());
        return node;
    }
    catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
ArrayType: ") + e.what());
    }
}
```

### 3.11 Implementasi Node Range

Fungsi `Range()` merupakan fungsi pengecekan terhadap rentang nilai dalam bahasa Arion. Fungsi ini memvalidasi struktur range yang terdiri dari dua expression yang dipisahkan oleh dua token period (`..`). Parser akan memanggil `Expression()` untuk membaca batas awal dan batas akhir range. Dengan penggunaan `Expression()`, bentuk batas range menjadi lebih fleksibel karena tidak hanya terbatas pada constant sederhana.

```
TreeParser* Parser::Range() {
    TreeParser* node = new TreeParser("<range>");
    try {
        node->addChild(Parser::Expression());
        node->addChild(Parser::match(period));
        node->addChild(Parser::match(period));
        node->addChild(Parser::Expression());
        return node;
    }
    catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
Range: ") + e.what());
    }
}
```

### 3.12 Implementasi Node Enumerated

Fungsi Enumerated() merupakan fungsi pengecekan terhadap enumerated type pada bahasa Arion. Fungsi ini memvalidasi daftar identifier yang berada di dalam tanda kurung dan dipisahkan oleh token comma. Parsing dilakukan secara berulang hingga seluruh identifier berhasil diproses dan ditemukan token penutup rparent. Fungsi akan mengembalikan node <enumerated> apabila struktur sesuai grammar. Jika terjadi kesalahan sintaks, maka fungsi akan melempar std::runtime\_error.

```
TreeParser* Parser::Enumerated() {
    TreeParser* node = new TreeParser("<enumerated>");
    try {
        node->addChild(Parser::match(lparent));
        node->addChild(Parser::match(ident));
        while (Parser::check(comma)) {
            node->addChild(Parser::match(comma));
            node->addChild(Parser::match(ident));
        }
        node->addChild(Parser::match(rparent));
        return node;
    }
    catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
Enumerated: ") + e.what());
    }
}
```

### 3.13 Implementasi Node Record-type

Fungsi RecordType() merupakan fungsi pengecekan terhadap deklarasi record dalam bahasa Arion. Fungsi ini memvalidasi struktur record yang diawali keyword record, diikuti field list, dan diakhiri keyword end. Parser akan memanggil fungsi FieldList() untuk memproses seluruh field yang terdapat di dalam record. Jika struktur deklarasi record tidak sesuai grammar, maka fungsi akan menghasilkan std::runtime\_error.

```
TreeParser* Parser::RecordType() {
    TreeParser* node = new TreeParser("<record-type>");
    try {
        node->addChild(Parser::match(recordsy));
        node->addChild(Parser::FieldList());
        node->addChild(Parser::match(endsy));
        return node;
    }
    catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
RecordType: ") + e.what());
    }
}
```

```

    }
}

```

### 3.14 Implementasi Node Field-list

Fungsi FieldList() merupakan fungsi pengecekan terhadap daftar field pada record. Fungsi ini akan memvalidasi satu atau lebih field-part yang dipisahkan oleh token semicolon. Parsing dilakukan secara berulang selama masih ditemukan field berikutnya berupa identifier. Node <field-list> akan dikembalikan jika struktur field valid. Jika terjadi kesalahan sintaks, maka fungsi akan melempar `std::runtime_error`.

```

TreeParser* Parser::FieldList() {
    TreeParser* node = new TreeParser("<field-list>");
    try {
        node->addChild(Parser::FieldPart());
        while (Parser::check(semicolons)) {
            node->addChild(Parser::match(semicolons));
            if (Parser::check(ident)) {
                node->addChild(Parser::FieldPart());
            } else {
                break;
            }
        }
        return node;
    }
    catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
FieldList: ") + e.what());
    }
}

```

### 3.15 Implementasi Node Field-part

Fungsi FieldPart() merupakan fungsi pengecekan terhadap satu bagian field pada record. Fungsi ini memvalidasi struktur field yang terdiri dari daftar identifier, token colon, dan tipe data field. Parser akan memanggil IdentifierList() untuk membaca satu atau lebih nama field, kemudian mencocokkan token colon, dan memanggil Type() untuk membaca tipe data dari field tersebut. Jika struktur field tidak sesuai, maka fungsi akan menghasilkan `std::runtime_error`.

```

TreeParser* Parser::FieldPart() {
    TreeParser* node = new TreeParser("<field-part>");
    try {
        node->addChild(Parser::IdentifierList());
        node->addChild(Parser::match(colon));
    }
}

```

```

        node->addChild(Parser::Type());
        return node;
    }
    catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
FieldPart: ") + e.what());
    }
}

```

### 3.16 Implementasi Node Var-declaration

Fungsi `VarDeclaration()` merupakan fungsi pengecekan pada deklarasi variabel dalam bahasa Arion. Fungsi ini memvalidasi deklarasi yang diawali keyword `var`, diikuti daftar identifier, token colon, tipe data, dan token semicolon. Parsing dilakukan secara berulang selama masih ditemukan identifier baru. Fungsi akan mengembalikan node `<var-declaration>` apabila seluruh deklarasi variabel valid. Jika terjadi kesalahan sintaks, maka fungsi akan melempar `std::runtime_error`.

```

TreeParser* Parser::VarDeclaration() {
    TreeParser* node = new TreeParser("<var-declaration>");
    try {
        node->addChild(Parser::match(varsy));
        do {
            node->addChild(Parser::IdentifierList());
            node->addChild(Parser::match(colon));
            node->addChild(Parser::Type());
            node->addChild(Parser::match(semicolon));
        } while (Parser::check(ident));
        return node;
    }
    catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
VarDeclaration: ") + e.what());
    }
}

```

### 3.17 Implementasi Node Identifier-list

Fungsi `IdentifierList()` merupakan fungsi pengecekan terhadap daftar identifier yang dipisahkan oleh token comma. Fungsi ini akan melakukan parsing identifier pertama, kemudian memproses identifier berikutnya selama masih ditemukan token comma. Fungsi akan mengembalikan node `<identifier-list>` apabila struktur sesuai grammar. Jika format daftar identifier tidak valid, maka fungsi akan menghasilkan `std::runtime_error`.

```

TreeParser* Parser::IdentifierList() {

```

```

TreeParser* node = new TreeParser("<identifier-list>");
try {
    node->addChild(Parser::match(ident));
    while (Parser::check(comma)) {
        node->addChild(Parser::match(comma));
        node->addChild(Parser::match(ident));
    }
    return node;
}
catch (const std::runtime_error& e) {
    throw std::runtime_error(std::string("Syntax error in
IdentifierList: ") + e.what());
}
}

```

### 3.18 Implementasi Node Subprogram-declaration

Fungsi SubprogramDeclaration() merupakan fungsi pengecekan terhadap deklarasi subprogram dalam bahasa Arion. Fungsi ini digunakan untuk menentukan apakah subprogram yang diproses berupa procedure atau function. Pemeriksaan dilakukan berdasarkan token awal, yaitu proceduresy atau functionsy. Setelah jenis subprogram diketahui, parser akan memanggil fungsi parser yang sesuai, yaitu ProcedureDeclaration() atau FunctionDeclaration(). Jika token awal tidak sesuai dengan kedua jenis subprogram tersebut, maka fungsi akan melempar std::runtime\_error.

```

TreeParser* Parser::SubprogramDeclaration() {
    TreeParser* node = new TreeParser("<subprogram-declaration>");
    try {
        if (Parser::check(proceduresy)) {
            node->addChild(Parser::ProcedureDeclaration());
        } else if (Parser::check(functionsy)) {
            node->addChild(Parser::FunctionDeclaration());
        } else {
            throw std::runtime_error("expected procedure or
function keyword");
        }
        return node;
    }
    catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
SubprogramDeclaration: ") + e.what());
    }
}

```

### 3.19 Implementasi Node Procedure-declaration

Fungsi `ProcedureDeclaration()` merupakan fungsi pengecekan terhadap deklarasi procedure dalam bahasa Arion. Fungsi ini memvalidasi struktur procedure yang terdiri dari keyword `procedure`, identifier nama procedure, parameter formal opsional, token `semicolon`, block procedure, dan token `semicolon` penutup. Jika terdapat parameter formal, parser akan memanggil fungsi `FormalParameterList()`. Selanjutnya parser akan memproses isi procedure menggunakan fungsi `Block()`. Jika struktur procedure tidak sesuai grammar, maka fungsi akan menghasilkan `std::runtime_error`.

```
TreeParser* Parser::ProcedureDeclaration() {
    TreeParser* node = new TreeParser("<procedure-declaration>");
    try {
        node->addChild(Parser::match(proceduresy));
        node->addChild(Parser::match(ident));
        if (Parser::check(lparg)) {
            node->addChild(Parser::FormalParameterList());
        }
        node->addChild(Parser::match(semicol));
        node->addChild(Parser::Block());
        node->addChild(Parser::match(semicol));
        return node;
    }
    catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
ProcedureDeclaration: ") + e.what());
    }
}
```

### 3.20 Implementasi Node Function-declaration

Fungsi `FunctionDeclaration()` merupakan fungsi pengecekan terhadap deklarasi function dalam bahasa Arion. Struktur function terdiri dari keyword `function`, identifier nama function, parameter formal opsional, tipe nilai kembalian, block function, dan token `semicolon` penutup. Fungsi akan memvalidasi tipe return melalui token identifier setelah token `colon`. Isi function kemudian diproses menggunakan parser `Block()`. Jika struktur deklarasi function tidak sesuai grammar, maka fungsi akan melempar `std::runtime_error`.

```
TreeParser* Parser::FunctionDeclaration() {
    TreeParser* node = new TreeParser("<function-declaration>");
    try {
        node->addChild(Parser::match(functionsy));
        node->addChild(Parser::match(ident));
        if (Parser::check(lparg)) {
            node->addChild(Parser::FormalParameterList());
        }
    }
}
```

```

    }
    node->addChild(Parser::match(colon));
    node->addChild(Parser::match(ident));
    node->addChild(Parser::match(semicolon));
    node->addChild(Parser::Block());
    node->addChild(Parser::match(semicolon));
    return node;
}
catch (const std::runtime_error& e) {
    throw std::runtime_error(std::string("Syntax error in
FunctionDeclaration: ") + e.what());
}
}

```

### 3.21 Implementasi Node Block

Fungsi Block() merupakan fungsi pengecekan terhadap blok program dalam bahasa Arion. Fungsi ini memvalidasi bahwa sebuah block terdiri dari bagian deklarasi (declaration-part) dan compound statement (compound-statement). Parser akan memanggil fungsi DeclarationPart() untuk memproses seluruh deklarasi di dalam block, kemudian memanggil CompoundStatement() untuk memproses statement utama program. Jika struktur block tidak valid, maka fungsi akan menghasilkan std::runtime\_error.

```

TreeParser* Parser::Block() {
    TreeParser* node = new TreeParser("<block>");
    try {
        node->addChild(Parser::DeclarationPart());
        node->addChild(Parser::CompoundStatement());
        return node;
    }
    catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
Block: ") + e.what());
    }
}

```

### 3.22 Implementasi Node Formal-parameter-list

Fungsi FormalParameterList() merupakan fungsi pengecekan terhadap daftar parameter formal pada procedure atau function. Fungsi ini memvalidasi struktur parameter yang berada di dalam tanda kurung dan dipisahkan menggunakan token semicolon. Setiap kelompok parameter akan diproses menggunakan fungsi ParameterGroup(). Parsing dilakukan secara berulang hingga ditemukan token penutup rparent. Jika struktur parameter formal tidak sesuai grammar, maka fungsi akan melempar std::runtime\_error.

```

TreeParser* Parser::FormalParameterList() {
    TreeParser* node = new TreeParser("<formal-parameter-list>");
    try {
        node->addChild(Parser::match(lparent));
        node->addChild(Parser::ParameterGroup());
        while (Parser::check(semicolon)) {
            node->addChild(Parser::match(semicolon));
            node->addChild(Parser::ParameterGroup());
        }
        node->addChild(Parser::match(rparent));
        return node;
    }
    catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
FormalParameterList: ") + e.what());
    }
}

```

### 3.23 Implementasi Node Parameter-group

Fungsi ParameterGroup() merupakan fungsi pengecekan terhadap satu kelompok parameter formal dalam bahasa Arion. Fungsi ini memvalidasi daftar identifier parameter, token colon, serta tipe parameter. Parser akan memanggil IdentifierList() untuk membaca satu atau lebih nama parameter. Setelah itu, parser mencocokkan token colon. Jika tipe parameter berupa array, parser akan memanggil ArrayType(), sedangkan tipe biasa akan diproses menggunakan token identifier.

```

TreeParser* Parser::ParameterGroup() {
    TreeParser* node = new TreeParser("<parameter-group>");
    try {
        node->addChild(Parser::IdentifierList());
        node->addChild(Parser::match(colon));
        if (Parser::check(array)) {
            node->addChild(Parser::ArrayType());
        } else {
            node->addChild(Parser::match(ident));
        }
        return node;
    }
    catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
ParameterGroup: ") + e.what());
    }
}

```

### 3.24 Implementasi Node Compound-statement

Fungsi CompoundStatement() merupakan fungsi pengecekan terhadap blok statement utama pada bahasa Arion. Fungsi ini memvalidasi struktur yang diawali keyword begin, diikuti daftar statement, dan diakhiri keyword end. Parser akan memanggil fungsi StatementList() untuk memproses seluruh statement di dalam blok tersebut. Jika struktur compound statement tidak sesuai grammar, maka fungsi akan menghasilkan `std::runtime_error`.

```
TreeParser* Parser::CompoundStatement() {
    TreeParser* node = new TreeParser("<compound-statement>");
    try {
        node->addChild(match(beginsy));
        node->addChild(StatementList());
        node->addChild(match(endsy));
        return node;
    } catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
<compound-statement>: ") + e.what());
    }
}
```

### 3.25 Implementasi Node Statement-list

Fungsi StatementList() merupakan fungsi pengecekan terhadap daftar statement dalam bahasa Arion. Fungsi ini memvalidasi satu atau lebih statement yang dipisahkan oleh token semicolon. Parser akan memanggil fungsi Statement() untuk memproses setiap statement secara berurutan. Parsing dilakukan berulang selama masih ditemukan token semicolon. Jika struktur daftar statement tidak sesuai grammar, maka fungsi akan melempar `std::runtime_error`.

```
TreeParser* Parser::StatementList() {
    TreeParser* node = new TreeParser("<statement-list>");
    try {
        node->addChild(Statement());
        while (check(semicolon)) {
            node->addChild(match(semicolon));
            node->addChild(Statement());
        }
        return node;
    } catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
<statement-list>: ") + e.what());
    }
}
```

### 3.26 Implementasi Node Statement

Fungsi Statement() merupakan fungsi pengecekan terhadap berbagai jenis statement dalam bahasa Arion. Fungsi ini menentukan jenis statement berdasarkan token awal yang ditemukan, seperti assignment statement, if statement, case statement, while statement, repeat statement, for statement, procedure/function call, maupun compound statement. Untuk identifier, parser akan membedakan antara assignment dan procedure/function call dengan melihat token berikutnya. Setelah jenis statement diketahui, parser akan memanggil fungsi parser yang sesuai. Jika terjadi kesalahan sintaks pada statement, maka fungsi akan menghasilkan `std::runtime_error`.

```
TreeParser* Parser::Statement() {
    TreeParser* node = new TreeParser("<statement>");
    try {
        if (check(ident)) {
            if (checkNext(becomes) || checkNext(period) ||
checkNext(lbrack)) {
                node->addChild(AssignmentStatement());
            } else {
                node->addChild(ProcedureFunctionCall());
            }
        } else if (check(ifsy)) {
            node->addChild(IfStatement());
        } else if (check(casesy)) {
            node->addChild(CaseStatement());
        } else if (check(whilesy)) {
            node->addChild(WhileStatement());
        } else if (check(repeatsy)) {
            node->addChild(RepeatStatement());
        } else if (check(forsy)) {
            node->addChild(ForStatement());
        } else if (check(beginsy)) {
            node->addChild(CompoundStatement());
        }
        return node;
    } catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
<statement>: ") + e.what());
    }
}
```

### 3.27 Implementasi Node Assignment-statement

Fungsi AssignmentStatement() merupakan fungsi pengecekan terhadap statement penugasan nilai dalam bahasa Arion. Struktur assignment terdiri dari variable, operator assignment (`:=`), dan expression. Parser akan memanggil fungsi Variable() untuk memproses target assignment dan Expression() untuk

memproses nilai yang diberikan. Jika struktur assignment tidak sesuai grammar, maka fungsi akan melempar `std::runtime_error`.

```
TreeParser* Parser::AssignmentStatement() {
    TreeParser* node = new TreeParser("<assignment-statement>");
    try {
        node->addChild(Variable());
        node->addChild(match(becomes));
        node->addChild(Expression());
        return node;
    } catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
<assignment-statement>: ") + e.what());
    }
}
```

### 3.28 Implementasi Node If-statement

Fungsi `IfStatement()` merupakan fungsi pengecekan terhadap statement percabangan if dalam bahasa Arion. Fungsi ini memvalidasi struktur yang terdiri dari keyword if, expression kondisi, keyword then, statement utama, dan statement else opsional. Parser akan memanggil `Expression()` untuk memproses kondisi dan `Statement()` untuk memproses isi percabangan. Jika struktur statement if tidak sesuai grammar, maka fungsi akan menghasilkan `std::runtime_error`.

```
TreeParser* Parser::IfStatement() {
    TreeParser* node = new TreeParser("<if-statement>");
    try {
        node->addChild(match(ifsy));
        node->addChild(Expression());
        node->addChild(match(thensy));
        node->addChild(Statement());
        if (check(elsesy)) {
            node->addChild(match(elsesy));
            node->addChild(Statement());
        }
        return node;
    } catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
<if-statement>: ") + e.what());
    }
}
```

### 3.29 Implementasi Node Case-statement

Fungsi `CaseStatement()` merupakan fungsi pengecekan terhadap statement case dalam bahasa Arion. Struktur case terdiri dari keyword case, expression, keyword of, daftar case block, dan keyword

end. Parser akan memanggil Expression() untuk memproses expression selector dan CaseBlock() untuk memproses seluruh cabang case. Jika struktur case statement tidak valid, maka fungsi akan melempar `std::runtime_error`.

```
TreeParser* Parser::CaseStatement() {
    TreeParser* node = new TreeParser("<case-statement>");
    try {
        node->addChild(match(casesy));
        node->addChild(Expression());
        node->addChild(match(ofsy));
        node->addChild(CaseBlock());
        node->addChild(match(endsy));
        return node;
    } catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
<case-statement>: ") + e.what());
    }
}
```

### 3.30 Implementasi Node Case-block

Fungsi CaseBlock() merupakan fungsi pengecekan terhadap blok pada statement case. Fungsi ini memvalidasi daftar konstanta yang dipisahkan comma, token colon, dan statement yang akan dijalankan. Selain itu, parser juga mendukung beberapa case block yang dipisahkan token semicolon. Parsing dilakukan secara rekursif selama masih ditemukan case berikutnya. Jika struktur case block tidak sesuai grammar, maka fungsi akan menghasilkan `std::runtime_error`.

```
TreeParser* Parser::CaseBlock() {
    TreeParser* node = new TreeParser("<case-block>");
    try {
        node->addChild(Constant());
        while (check(comma)) {
            node->addChild(match(comma));
            node->addChild(Constant());
        }
        node->addChild(match(colon));
        node->addChild(Statement());

        while (check(semicolon)) {
            node->addChild(match(semicolon));
            bool hasNextCase = check(ident) || check(intcon) ||
check(realcon) ||
                                check(charcon) || check(stringcon)
||
                                check(plus) || check(minus);
            if (hasNextCase && !check(endsy)) {
                node->addChild(CaseBlock());
            }
        }
    } catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
<case-block>: ") + e.what());
    }
}
```

```

        }
        if (!hasNextCase) break;
    }
    return node;
} catch (const std::runtime_error& e) {
    throw std::runtime_error(std::string("Syntax error in
<case-block>: ") + e.what());
}
}

```

### 3.31 Implementasi Node While-statement

Fungsi WhileStatement() merupakan fungsi pengecekan terhadap statement perulangan while dalam bahasa Arion. Struktur statement ini terdiri dari keyword while, expression kondisi, keyword do, dan statement yang akan dijalankan selama kondisi bernilai benar. Parser akan memanggil fungsi Expression() untuk memproses kondisi perulangan dan Statement() untuk memproses isi perulangan. Jika struktur while statement tidak sesuai grammar, maka fungsi akan menghasilkan std::runtime\_error.

```

TreeParser* Parser::WhileStatement() {
    TreeParser* node = new TreeParser("<while-statement>");
    try {
        node->addChild(match(whilesy));
        node->addChild(Expression());
        node->addChild(match(dosy));
        node->addChild(Statement());
        return node;
    } catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
<while-statement>: ") + e.what());
    }
}

```

### 3.32 Implementasi Node Repeat-statement

Fungsi RepeatStatement() merupakan fungsi pengecekan terhadap statement perulangan repeat-until dalam bahasa Arion. Fungsi ini memvalidasi struktur yang terdiri dari keyword repeat, daftar statement, keyword until, dan expression kondisi penghentian. Parser akan memanggil fungsi StatementList() untuk memproses seluruh statement di dalam perulangan dan Expression() untuk memproses kondisi akhir perulangan. Jika struktur repeat statement tidak sesuai grammar, maka fungsi akan melempar std::runtime\_error.

```

TreeParser* Parser::RepeatStatement() {

```

```

TreeParser* node = new TreeParser("<repeat-statement>");
try {
    node->addChild(match(repeatsy));
    node->addChild(StatementList());
    node->addChild(match(untilsy));
    node->addChild(Expression());
    return node;
} catch (const std::runtime_error& e) {
    throw std::runtime_error(std::string("Syntax error in
<repeat-statement>: ") + e.what());
}
}

```

### 3.33 Implementasi Node For-statement

Fungsi ForStatement() merupakan fungsi pengecekan terhadap statement perulangan for dalam bahasa Arion. Struktur statement terdiri dari keyword for, identifier counter, operator assignment (:=), expression awal, keyword to atau downto, expression akhir, keyword do, dan statement yang dijalankan pada setiap iterasi. Parser akan memvalidasi arah iterasi menggunakan token tosy atau downtosy, kemudian memproses expression batas iterasi dan isi statement menggunakan parser yang sesuai. Jika struktur for statement tidak valid, maka fungsi akan menghasilkan `std::runtime_error`.

```

TreeParser* Parser::ForStatement() {
    TreeParser* node = new TreeParser("<for-statement>");
    try {
        node->addChild(match(forsy));
        node->addChild(match(ident));
        node->addChild(match(becomes));
        node->addChild(Expression());
        if (check(tosy)) node->addChild(match(tosy));
        else if (check(downtosy)) node->addChild(match(downtosy));
        else throw std::runtime_error("expected to or downto
keyword in <for-statement>");
        node->addChild(Expression());
        node->addChild(match(dosy));
        node->addChild(Statement());
        return node;
    } catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
<for-statement>: ") + e.what());
    }
}

```

### 3.34 Implementasi Node Procedure/function-call

Fungsi `ProcedureFunctionCall()` merupakan fungsi pengecekan terhadap pemanggilan procedure atau function dalam bahasa Arion. Struktur pemanggilan terdiri dari identifier nama procedure/function dan daftar parameter aktual opsional yang berada di dalam tanda kurung. Jika ditemukan token `lparent`, parser akan memanggil fungsi `ParameterList()` untuk memproses seluruh parameter aktual. Fungsi akan mengembalikan node `<procedure/function-call>` apabila struktur pemanggilan valid. Jika terjadi kesalahan sintaks, maka fungsi akan melempar `std::runtime_error`.

```
TreeParser* Parser::ProcedureFunctionCall() {
    TreeParser* node = new TreeParser("<procedure/function-call>");
    try {
        node->addChild(match(ident));
        if (check(lparent)) {
            node->addChild(match(lparent));
            if (!check(rparent)) {
                node->addChild(ParameterList());
            }
            node->addChild(match(rparent));
        }
        return node;
    } catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
<procedure/function-call>: ") + e.what());
    }
}
```

### 3.35 Implementasi Node Parameter-list

Fungsi `ParameterList()` merupakan fungsi pengecekan terhadap daftar parameter aktual pada pemanggilan procedure atau function. Fungsi ini memvalidasi satu atau lebih expression yang dipisahkan oleh token comma. Parser akan memanggil fungsi `Expression()` untuk memproses setiap parameter aktual secara berurutan. Parsing dilakukan secara berulang selama masih ditemukan token comma. Jika struktur parameter list tidak sesuai grammar, maka fungsi akan menghasilkan `std::runtime_error`.

```
TreeParser* Parser::ParameterList() {
    TreeParser* node = new TreeParser("<parameter-list>");
    try {
        node->addChild(Expression());
        while (check(comma)) {
            node->addChild(match(comma));
            node->addChild(Expression());
        }
        return node;
    } catch (const std::runtime_error& e) {
```

```

        throw std::runtime_error(std::string("Syntax error in
<parameter-list>: ") + e.what());
    }
}

```

### 3.36 Implementasi Node Variable

Fungsi Variable() merupakan fungsi pengecekan terhadap variabel dalam bahasa Arion. Variabel selalu diawali dengan token ident. Setelah identifier berhasil dibaca, parser akan mengecek apakah terdapat akses lanjutan berupa array index atau field record. Jika setelah identifier ditemukan token lbrack atau period, parser akan memanggil ComponentVariable() secara berulang untuk membaca akses komponen tersebut. Dengan cara ini, variable dapat berbentuk identifier biasa, elemen array, field record, maupun gabungan akses bertingkat.

```

TreeParser* Parser::Variable() {
    TreeParser* node = new TreeParser("<variable>");
    try {
        node->addChild(match(ident));
        while (check(lbrack) || check(period)) {
            node->addChild(ComponentVariable());
        }
        return node;
    } catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
<variable>: ") + e.what());
    }
}

```

### 3.37 Implementasi Node Component-variable

Fungsi ComponentVariable() merupakan fungsi pengecekan terhadap akses lanjutan pada variabel. Akses lanjutan tersebut dapat berupa indeks array yang diawali token lbrack dan diakhiri token rbrack, atau akses field record yang diawali token period dan diikuti token ident. Fungsi ini dipanggil setelah identifier utama pada Variable() berhasil dibaca, sehingga ComponentVariable() hanya menangani bagian lanjutan dari suatu variabel.

```

TreeParser* Parser::ComponentVariable() {
    TreeParser* node = new TreeParser("<component-variable>");
    try {
        if (check(lbrack)) {
            node->addChild(match(lbrack));
            node->addChild(IndexList());
            node->addChild(match(rbrack));
        } else if (check(period)) {

```

```

        node->addChild(match(period));
        node->addChild(match(ident));
    } else {
        throw std::runtime_error("expected array index or
record field access");
    }
    return node;
} catch (const std::runtime_error& e) {
    throw std::runtime_error(std::string("Syntax error in
<component-variable>: ") + e.what());
}
}

```

### 3.38 Implementasi Node Index-list

Fungsi `IndexList()` merupakan fungsi pengecekan terhadap daftar indeks array dalam bahasa Arion. Fungsi ini memvalidasi indeks berupa integer constant, character constant, maupun identifier. Selain itu, parser juga mendukung beberapa indeks yang dipisahkan menggunakan token comma. Parsing dilakukan secara rekursif hingga seluruh indeks berhasil diproses. Jika struktur index list tidak sesuai grammar, maka fungsi akan melempar `std::runtime_error`.

```

TreeParser* Parser::IndexList() {
    TreeParser* node = new TreeParser("<index-list>");
    try {
        if (check(intcon)) {
            node->addChild(match(intcon));
        } else if (check(charcon)) {
            node->addChild(match(charcon));
        } else if (check(ident)) {
            node->addChild(match(ident));
        } else {
            throw std::runtime_error("expected integer, character,
or identifier in array index");
        }

        if (check(comma)) {
            node->addChild(match(comma));
            node->addChild(IndexList());
        }

        return node;
    } catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
<index-list>: ") + e.what());
    }
}

```

### 3.39 Implementasi Node Expression

Fungsi Expression() merupakan fungsi pengecekan terhadap expression dalam bahasa Arion. Struktur expression terdiri dari simple expression yang dapat diikuti relational operator dan simple expression lainnya. Parser akan memanggil fungsi SimpleExpression() untuk memproses operand, kemudian memanggil RelationalOperator() apabila ditemukan operator relasional seperti ==, <, >, <=, >=, atau <=. Jika struktur expression tidak valid, maka fungsi akan menghasilkan std::runtime\_error.

```
TreeParser* Parser::Expression() {
    TreeParser* node = new TreeParser("<expression>");
    try {
        node->addChild(SimpleExpression());
        if (check(eql) || check(neq) || check(gtr) ||
            check(geq) || check(lss) || check(leq)) {
            node->addChild(RelationalOperator());
            node->addChild(SimpleExpression());
        }
        return node;
    } catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
<expression>: ") + e.what());
    }
}
```

### 3.40 Implementasi Node Simple-expression

Fungsi SimpleExpression() merupakan fungsi pengecekan terhadap expression tanpa operator relasional dalam bahasa Arion. Fungsi ini mendukung unary operator plus dan minus, serta additive operator seperti +, -, dan or. Parser akan memproses term pertama menggunakan Term(), kemudian melanjutkan parsing secara berulang selama masih ditemukan additive operator. Jika struktur simple expression tidak sesuai grammar, maka fungsi akan melempar std::runtime\_error.

```
TreeParser* Parser::SimpleExpression() {
    TreeParser* node = new TreeParser("<simple-expression>");
    try {
        if (check(plus)) node->addChild(match(plus));
        else if (check(minus)) node->addChild(match(minus));

        node->addChild(Term());
        while (check(plus) || check(minus) || check(orsy)) {
            node->addChild(AdditiveOperator());
            node->addChild(Term());
        }
        return node;
    } catch (const std::runtime_error& e) {
```

```

        throw std::runtime_error(std::string("Syntax error in
<simple-expression>: ") + e.what());
    }
}

```

### 3.41 Implementasi Node Term

Fungsi Term() merupakan fungsi pengecekan terhadap bagian expression dengan prioritas operasi lebih tinggi dalam bahasa Arion. Fungsi ini memvalidasi factor yang dapat diikuti multiplicative operator seperti \*, /, div, mod, dan and. Parser akan memanggil Factor() untuk memproses operand dan MultiplicativeOperator() untuk memproses operator. Parsing dilakukan secara berulang selama masih ditemukan operator multiplicative. Jika struktur term tidak valid, maka fungsi akan menghasilkan std::runtime\_error.

```

TreeParser* Parser::Term() {
    TreeParser* node = new TreeParser("<term>");
    try {
        node->addChild(Factor());
        while (check(times) || check(rdiv) || check(idiv) ||
               check(imod) || check(andsy)) {
            node->addChild(MultiplicativeOperator());
            node->addChild(Factor());
        }
        return node;
    } catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
<term>: ") + e.what());
    }
}

```

### 3.42 Implementasi Node Factor

Fungsi Factor() merupakan fungsi pengecekan terhadap unit terkecil expression dalam bahasa Arion. Factor dapat berupa variable, literal integer, real, karakter, string, expression di dalam tanda kurung, unary operator not, maupun procedure/function call. Parser akan menentukan jenis factor berdasarkan token awal yang ditemukan dan memanggil parser yang sesuai. Jika token yang ditemukan tidak termasuk bentuk factor yang valid, maka fungsi akan melempar std::runtime\_error beserta informasi token yang menyebabkan kesalahan sintaks.

```

TreeParser* Parser::Factor() {
    TreeParser* node = new TreeParser("<factor>");
    try {
        if (check(ident)) {

```

```

        if (checkNext(lpnext))
node->addChild(ProcedureFunctionCall());
        else
            node->addChild(Variable());
    } else if (check(intcon)) {
        node->addChild(match(intcon));
    } else if (check(realcon)) {
        node->addChild(match(realcon));
    } else if (check(charcon)) {
        node->addChild(match(charcon));
    } else if (check(stringcon)) {
        node->addChild(match(stringcon));
    } else if (check(lpnext)) {
        node->addChild(match(lpnext));
        node->addChild(Expression());
        node->addChild(match(rpnext));
    } else if (check(notsy)) {
        node->addChild(match(notsy));
        node->addChild(Factor());
    } else {
        throw std::runtime_error(
            "invalid token in <factor>: found " +
            tokenToErrorString(currentToken()));
    }
    return node;
} catch (const std::runtime_error& e) {
    throw std::runtime_error(std::string("Syntax error in
<factor>: ") + e.what());
}
}

```

### 3.43 Implementasi Node Relational-operator

Fungsi RelationalOperator() merupakan fungsi pengecekan terhadap operator relasional dalam bahasa Arion. Fungsi ini memvalidasi operator perbandingan seperti ==, <, >, >=, <=, dan <=. Parser akan melakukan matching terhadap token operator yang ditemukan dan menambahkan token tersebut sebagai child node. Jika token bukan operator relasional yang valid, maka fungsi akan menghasilkan std::runtime\_error.

```

TreeParser* Parser::RelationalOperator() {
    TreeParser* node = new TreeParser("<relational-operator>");
    try {
        if (check(eql)) node->addChild(match(eql));
        else if (check(neq)) node->addChild(match(neq));
        else if (check(gtr)) node->addChild(match(gtr));
        else if (check(geq)) node->addChild(match(geq));
        else if (check(lss)) node->addChild(match(lss));
        else if (check(leq)) node->addChild(match(leq));
    }
}

```

```

        else throw std::runtime_error("expected relational
operator: ==, <>, >, >=, <, or <=");
        return node;
    } catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
<relational-operator>: ") + e.what());
    }
}

```

### 3.44 Implementasi Node Additive-operator

Fungsi AdditiveOperator() merupakan fungsi pengecekan terhadap operator additive dalam bahasa Arion. Operator yang didukung meliputi +, -, dan keyword or. Parser akan melakukan matching terhadap operator yang ditemukan dan menambahkannya ke parse tree. Jika token yang ditemukan bukan operator additive yang valid, maka fungsi akan melempar std::runtime\_error.

```

TreeParser* Parser::AdditiveOperator() {
    TreeParser* node = new TreeParser("<additive-operator>");
    try {
        if (check(plus)) node->addChild(match(plus));
        else if (check(minus)) node->addChild(match(minus));
        else if (check(orsy)) node->addChild(match(orsy));
        else throw std::runtime_error("expected additive operator:
+, -, or or");
        return node;
    } catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
<additive-operator>: ") + e.what());
    }
}

```

### 3.45 Implementasi Node Multiplicative-operator

Fungsi MultiplicativeOperator() merupakan fungsi pengecekan terhadap operator multiplicative dalam bahasa Arion. Operator yang didukung meliputi \*, /, div, mod, dan and. Parser akan melakukan matching terhadap token operator yang ditemukan dan menambahkannya ke parse tree sebagai child node. Jika token tidak sesuai dengan operator multiplicative yang valid, maka fungsi akan menghasilkan std::runtime\_error.

```

TreeParser* Parser::MultiplicativeOperator() {
    TreeParser* node = new TreeParser("<multiplicative-operator>");
    try {
        if (check(times)) node->addChild(match(times));
        else if (check(rdiv)) node->addChild(match(rdiv));
        else if (check(idiv)) node->addChild(match(idiv));
    }
}

```

```

        else if (check(imod)) node->addChild(match(imod));
        else if (check(andsy)) node->addChild(match(andsy));
        else throw std::runtime_error("expected multiplicative
operator: *, /, div, mod, or and");
        return node;
    } catch (const std::runtime_error& e) {
        throw std::runtime_error(std::string("Syntax error in
<multiplicative-operator>: ") + e.what());
    }
}

```

### 3.46 Implementasi Program Pencetakan Parse Tree

Program pencetakan Parse Tree hasil parsing diimplementasikan melalui fungsi `printParseTree()`. Fungsi ini bertugas untuk mencetak struktur Parse Tree secara hierarkis ke standard output dan juga menyimpannya ke file output yang diberikan pengguna. Struktur tree divisualisasikan menggunakan karakter ASCII seperti `└─`, `└─`, dan `|` sehingga hubungan antar node dapat terlihat dengan jelas.

Dalam implementasinya, fungsi `printParseTree()` memanfaatkan dua helper function, yaitu `writeParseTree()` dan `printParseTreeLine()`. Fungsi `writeParseTree()` digunakan untuk mencetak root node dan memulai traversal terhadap child node, sedangkan fungsi `printParseTreeLine()` digunakan untuk melakukan traversal rekursif terhadap seluruh node pada tree dan menentukan format indentasi setiap node berdasarkan posisinya di dalam tree.

Sebelum melakukan pencetakan, fungsi `printParseTree()` akan melakukan validasi apakah Parse Tree kosong (`nullptr`) dan memastikan direktori output tersedia menggunakan library `std::filesystem`. Jika file output gagal dibuka atau Parse Tree kosong, maka fungsi akan menghasilkan `std::runtime_error`.

```

// Fungsi Utama
static void printParseTreeLine(TreeParser* node, std::ostream&
output, const std::string& prefix, bool isLast) {
    output << prefix;
    if (isLast) {
        output << "└─ ";
    } else {
        output << "|─ ";
    }
    output << node->data << '\n';

    std::string childPrefix = prefix;
    if (isLast) {

```

```

        childPrefix += "    ";
    } else {
        childPrefix += "|    ";
    }
    for (size_t i = 0; i < node->children.size(); ++i) {
        printParseTreeLine(node->children[i], output, childPrefix,
i == node->children.size() - 1);
    }
}

// Fungsi Helper writeParseTree()
static void writeParseTree(std::ostream& output, TreeParser* root)
{
    output << root->data << '\n';
    for (size_t i = 0; i < root->children.size(); ++i) {
        printParseTreeLine(root->children[i], output, "", i ==
root->children.size() - 1);
    }
}

// Fungsi Helper printParseTree()
void Parser::printParseTree(TreeParser* node, const std::string&
path) const {
    if (node == nullptr) {
        throw std::runtime_error("empty parse tree");
    }

    std::filesystem::path outputPath(path);
    if (outputPath.has_parent_path()) {
std::filesystem::create_directories(outputPath.parent_path());
    }

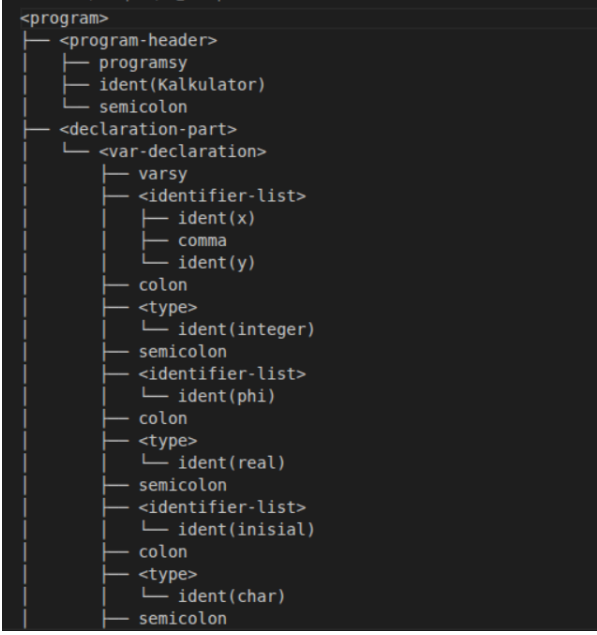
    std::ofstream file(path);
    if (!file.is_open()) {
        throw std::runtime_error("failed to open output file: " +
path);
    }

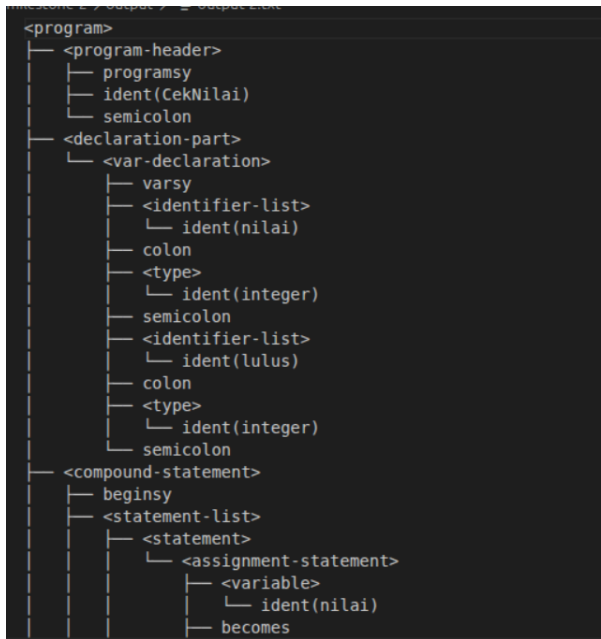
    writeParseTree(std::cout, node);
    writeParseTree(file, node);
}

```

## BAB 4 Pengujian

### 4.1 Pengujian Masukkan dan Keluaran Program

| Masukkan                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | Keluaran                                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Test 1 - Nama File : input-1.txt</b><br/>         {pengujian 1 - bagian tipe data primitif dan operator aritmatika}</p> <pre> program Kalkulator; var   x, y      : integer;   phi       : real;   inisial   : char;   pesan     : string;   hasil     : integer; begin   (* isi nilai awal untuk setiap variabel *)   x         := 20;   y         := 7;   phi       := 3.14;   inisial   := 'A';   pesan     := 'Halo Arion';    { hitung hasil penjumlahan, pengurangan,   dan perkalian }   hasil := x + y;   hasil := x - y;   hasil := x * y;    (* pembagian real: 22 dibagi 7 mendekati   nilai phi *)   phi := 22 / 7;    { tampilkan semua hasil ke layar }   writeln(hasil);   writeln(phi);   writeln(inisial);   writeln(pesan); end.</pre> | <p><b>Keluaran File :output-1.txt</b></p>  <p>Seterusnya dapat dilihat di repository pada lampiran</p> |
| <p><b>Test 2 - Nama File :input-2.txt</b><br/>         (* pengujian 2 - bagian percabangan dan operator logika *)</p> <pre> program CekNilai;</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | <p><b>Keluaran File :output-2.txt</b></p>                                                                                                                                                 |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                                                                                                                                                |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> var     nilai : integer;     lulus : integer; begin     nilai := 75;     lulus := 0;      { bandingkan nilai dengan beberapa ambang batas menggunakan operator relasional }     if nilai &gt; 90 then         writeln('Sangat Baik')     else if nilai &gt;= 75 then         writeln('Baik')     else if nilai &lt; 50 then         writeln('Kurang')     else if nilai &lt;= 59 then         writeln('Cukup')     else         writeln('Sedang');      (* pastikan nilai tidak sama dengan nol sebelum diproses lebih lanjut *)     if nilai &lt;&gt; 0 then         writeln('Nilai tidak nol');      { operator NOT membalik nilai logika dari kondisi yang dibungkus kurang }     if NOT (nilai &lt; 0) then         lulus := 1;      (* operator AND mensyaratkan kedua sisi bernilai benar secara bersamaan *)     if (nilai &gt;= 75) AND (nilai &lt;= 100) then         writeln('Lulus');      { cek apakah nilai di luar rentang yang diperbolehkan }     if (nilai &lt; 0) OR (nilai &gt; 100) then         writeln('Nilai tidak valid');      (* operator == diuji di sini, perhatikan jika ada bug pada EQL_STATE di lexer *)     if nilai == 75 then         writeln('Tepat 75'); end. </pre> |  <p>Seterusnya dapat dilihat di repository pada lampiran</p> |
| <p><b>Test 3 - Nama File :input-3.txt</b><br/> {pengujian 3 - bagian perulangan dengan for, while, dan repeat}</p> <p>program Perulangan;</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | <p><b>Keluaran File :output-3.txt</b></p>                                                                                                      |

```

var
    i, n, total : integer;
begin
    n      := 5;
    total := 0;

    { jumlahkan angka 1 sampai n menggunakan
    perulangan for naik }
    for i := 1 to n do
        total := total + i;

    writeln(total);

    (* cetak angka dari n turun ke 1
    menggunakan for downto *)
    for i := n downto 1 do
        writeln(i);

    { perulangan while akan terus berjalan
    selama kondisinya benar }
    i := 1;
    while i <= n do
    begin
        writeln(i);
        i := i + 1;
    end;

    (* repeat dijalankan minimal satu kali
    sebelum kondisi until diperiksa )
    i := 1;
    repeat
        writeln(i);
        i := i + 1;
    until i > n;
end.

```

```

<program>
├── <program-header>
│   ├── <programsy>
│   ├── <ident(Perulangan)>
│   └── <semicolon>
├── <declaration-part>
│   └── <var-declaration>
│       ├── <varsy>
│       │   └── <identifier-list>
│       │       ├── <ident(i)>
│       │       ├── <comma>
│       │       ├── <ident(n)>
│       │       ├── <comma>
│       │       └── <ident(total)>
│       ├── <colon>
│       └── <type>
│           └── <ident(integer)>
└── <semicolon>
├── <compound-statement>
│   ├── <beginsy>
│   └── <statement-list>
│       └── <statement>
│           └── <assignment-statement>
│               ├── <variable>
│               │   └── <ident(n)>
│               ├── <becomes>
│               ├── <expression>
│               └── <simple-expression>

```

Seterusnya dapat dilihat di repository pada lampiran

#### Test 4 - Nama File :input-4.txt

(\* pengujian 4 - bagian deklarasi tipe kompleks, fungsi, dan prosedur \*)

```

program TipeKompleks;
const
    MAX == 10;
    PI  == 3.14159;
type
    Titik == record
        x : real;
        y : real;
    end;
    Daftar == array [1..10] of integer;
var

```

#### Keluaran File :output-4.txt

```

p      : Titik;
data  : Daftar;
i      : integer;

{ fungsi ini menerima satu bilangan bulat dan
mengembalikan kuadratnya }
function Kuadrat(n : integer) : integer;
begin
    Kuadrat := n * n;
end;

(* prosedur ini mengisi array dengan nilai
kuadrat dari indeks nya *)
procedure IsiArray(arr : Daftar; n : integer);
var
    j : integer;
begin
    for j := 1 to n do
        arr[j] := j * j;
    end;

begin
    { isi koordinat titik dengan nilai real }
    p.x := 3.0;
    p.y := 4.0;

    (* panggil prosedur untuk mengisi data,
    lalu tampilkan hasilnya *)
    IsiArray(data, MAX);

    for i := 1 to MAX do
        writeln(data[i]);

    writeln(Kuadrat(5));
end.

```

```

<program>
├── <program-header>
│   ├── programsy
│   ├── ident(TipeKompleks)
│   └── semicolon
├── <declaration-part>
│   ├── <const-declaration>
│   │   ├── constsy
│   │   ├── ident(MAX)
│   │   ├── eql
│   │   ├── <constant>
│   │   │   └── intcon(10)
│   │   ├── semicolon
│   │   ├── ident(PI)
│   │   ├── eql
│   │   ├── <constant>
│   │   │   └── realcon(3.14159)
│   │   └── semicolon
│   ├── <type-declaration>
│   │   ├── typesy
│   │   ├── ident(Titik)
│   │   ├── eql
│   │   ├── <type>
│   │   │   └── <record-type>
│   │   │       ├── recordsy
│   │   │       ├── <field-list>
│   │   │       │   ├── <field-part>
│   │   │       │   │   └── <identifier-list>

```

Seterusnya dapat dilihat di repository pada lampiran

#### Test 5 - Nama File :input-5.txt

{pengujian 5 - bagian case, operator div dan mod, serta variasi komentar}

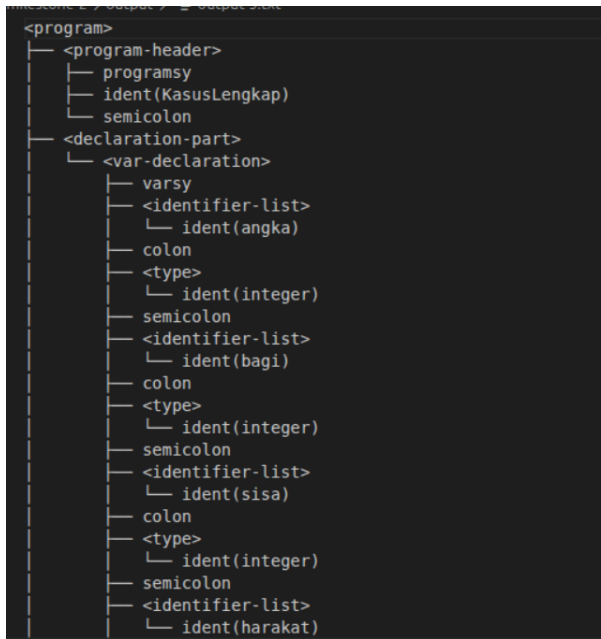
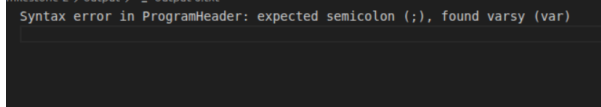
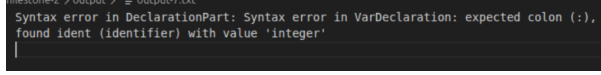
```

program KasusLengkap;
var
    angka    : integer;
    bagi     : integer;
    sisa     : integer;
    harakat  : char;
begin
    angka := 17;

    { bagi integer: 17 dibagi 5 hasilnya 3,
    bukan 3.4 }

```

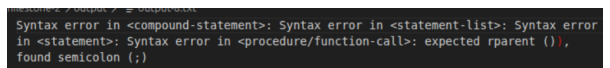
#### Keluaran File :output-5.txt

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |                                                                                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>         bagi := angka div 5;          (* sisa bagi: 17 mod 5 hasilnya 2 karena 5         masuk 3 kali dengan sisa 2 *)         sisa := angka MOD 5;          writeln(bagi);         writeln(sisa);          { gunakan case of untuk memeriksa nilai         sisa pembagian satu per satu }         case sisa of             0 : writeln('Habis dibagi 5');             1 : writeln('Sisa 1');             2 : writeln('Sisa 2');             3 : writeln('Sisa 3');             4 : writeln('Sisa 4');         end;          (* simpan satu karakter huruf ke dalam         variabel bertipe char *)         harakat := 'Z';          writeln(harakat);         end. </pre> |  <p>Seterusnya dapat dilihat di repository pada lampiran</p>                                  |
| <p><b>Test 6 - Nama File :input-6.txt</b><br/>         { pengujian 6 - error parsing karena titik koma setelah header program hilang }</p> <pre> program ErrorParsing var     x : integer; begin     x := 10; End. </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                         | <p><b>Keluaran File :output-6.txt</b></p>  <p>Dapat dilihat di repository pada lampiran</p> |
| <p><b>Test 7 - Nama File :input-7.txt</b><br/>         { pengujian 7 - error parsing karena titik dua pada deklarasi variabel hilang }</p> <pre> program ErrorDeklarasi; var     x integer; begin     x := 10; End. </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                        | <p><b>Keluaran File :output-7.txt</b></p>  <p>Dapat dilihat di repository pada lampiran</p> |

**Test 8 - Nama File :input-8.txt**

{ pengujian 8 - error parsing karena kurung  
tutup pada pemanggilan prosedur hilang }

```
program ErrorPemanggilan;  
begin  
    writeln('Halo Arion';  
end.
```

**Keluaran File :output-8.txt**A screenshot of a compiler's error output window. The text is white on a black background. It shows a multi-line error message: 'Syntax error in <compound-statement>: Syntax error in <statement-list>: Syntax error in <statement>: Syntax error in <procedure/function-call>: expected rparent (')', found semicolon (;)'.

Syntax error in <compound-statement>: Syntax error in <statement-list>: Syntax error in <statement>: Syntax error in <procedure/function-call>: expected rparent (')', found semicolon (;)

**Dapat dilihat di repository pada lampiran**

## **BAB 5**

### **Kesimpulan dan Saran**

#### **5.1 Kesimpulan**

Tahap syntax analysis merupakan langkah lanjutan yang sangat penting setelah proses lexical analysis dalam pembuatan compiler suatu bahasa pemrograman. Pada tahap ini, parser bertugas untuk menerima rangkaian token yang telah dihasilkan oleh lexer, kemudian memeriksa apakah susunan token tersebut sudah sesuai dengan aturan grammar bahasa pemrograman Arion. Proses ini tidak lagi membaca karakter satu per satu seperti pada lexer, tetapi membaca token sebagai satuan yang sudah memiliki makna leksikal. Parser kami dibuat dengan menerapkan konsep Recursive Descent Parser, yaitu metode parsing top-down yang memecah setiap aturan grammar menjadi fungsi-fungsi yang saling memanggil. Setiap fungsi pada parser merepresentasikan node atau bagian tertentu dalam grammar, seperti program, declaration-part, statement, expression, term, factor, dan node lainnya. Dengan pendekatan ini, parser dapat melakukan pengecekan sintaks secara bertahap dari struktur paling besar hingga bagian yang lebih kecil.

Keberhasilan implementasi parser dengan Recursive Descent memastikan bahwa proses pengecekan struktur program dapat dilakukan secara sistematis dan lebih mudah ditelusuri. Apabila susunan token sesuai dengan grammar, parser akan membentuk parse tree sebagai representasi hierarkis dari struktur program. Namun, apabila terdapat token yang tidak sesuai dengan aturan grammar, parser akan menghasilkan pesan error yang menunjukkan bahwa terjadi kesalahan sintaks pada bagian tertentu. Hasil dari syntax analysis ini memegang peranan penting untuk tahapan kompilasi berikutnya. Parse tree yang dihasilkan dapat digunakan sebagai dasar untuk proses semantic analysis, yaitu tahap pemeriksaan makna program seperti kesesuaian tipe data, deklarasi variabel, pemanggilan fungsi, dan aturan semantik lainnya. Dengan demikian, parser menjadi penghubung penting antara hasil lexical analysis dan tahap analisis program yang lebih lanjut.

#### **5.2 Saran**

Pada pengembangan berikutnya, parser masih dapat ditingkatkan agar mampu menangani struktur bahasa Arion dengan lebih lengkap dan lebih kuat. Salah satu peningkatan yang dapat dilakukan adalah memperjelas mekanisme penanganan error agar pesan kesalahan tidak hanya menunjukkan token yang tidak sesuai, tetapi juga memberikan informasi posisi baris dan kolom pada source code. Dengan adanya informasi tersebut, pengguna dapat lebih mudah menemukan dan memperbaiki kesalahan sintaks pada program yang dibuat.

Selain dari sisi parser, bagian lexer juga perlu dijaga agar hasil token yang diberikan kepada parser benar-benar konsisten. Hal ini penting karena parser sangat bergantung pada urutan dan jenis token yang dihasilkan oleh lexer. Jika lexer salah mengelompokkan karakter menjadi token, maka parser dapat gagal membaca struktur program meskipun source code sebenarnya sudah benar. Oleh karena itu, aturan tokenisasi pada lexer perlu dibuat lebih jelas, terutama untuk token yang memiliki bentuk mirip seperti period, range ..., assignment :=, equality ==, string, character, identifier, dan keyword.

Lexer juga sebaiknya tetap mengikuti prinsip pemisahan token yang jelas berdasarkan separator dan simbol. Misalnya, bentuk 1..10 harus dapat menghasilkan token yang sesuai untuk range, bukan terbaca sebagai real number yang rusak atau token unknown. Begitu juga identifier yang menyerupai keyword, seperti programku atau integerValue, harus tetap dikenali sebagai ident dan bukan dipotong menjadi keyword. Konsistensi seperti ini sangat penting agar parser tidak menerima token yang salah dan tidak menghasilkan syntax error yang sebenarnya berasal dari kesalahan lexer.

Selain itu, lexer sebaiknya tidak langsung menghentikan seluruh program ketika menemukan token yang tidak dikenal. Token unknown tetap dapat dihasilkan dengan informasi nilai token dan lokasi kemunculannya. Dengan begitu, parser atau program utama dapat memberikan pesan error yang lebih informatif kepada pengguna. Namun, token comment sebaiknya tetap dapat dibuang sebelum parsing karena komentar tidak mempengaruhi struktur sintaks program.

## Lampiran

**Link Github :** <https://github.com/Alpaomega1136/JML-Tubes-IF2224-2026.git>

**Pembagian Tugas :**

| NIM      | Nama                         | Pengerjaan pada milestone ini                     |
|----------|------------------------------|---------------------------------------------------|
| 13524015 | Mahatma Brahmana             | Membuat laporan milestone 2 & merevisi kode lexer |
| 13524057 | Benedict Darrel Setiawan     | Membuat laporan milestone 2 & merevisi kode lexer |
| 13524059 | Raymond Jonathan Dwi Putra J | Membuat laporan milestone 2 & membuat kode parser |
| 13524111 | Reynard Anderson Wijaya      | Membuat laporan milestone 2 & membuat kode parser |

## Daftar Pustaka

Tutorialspoint. (2014). *Parse Tree in Automata Theory*. Diakses dari:

[https://www.tutorialspoint.com/automata\\_theory/automata\\_theory\\_parse\\_tree.htm](https://www.tutorialspoint.com/automata_theory/automata_theory_parse_tree.htm)

GeeksForGeeks. (2026). *Parsing - Introduction to Parsers*. Diakses dari:

<https://www.geeksforgeeks.org/compiler-design/introduction-of-parsing-ambiguity-and-parsers-set-1/>

GeeksForGeeks. (2025). *Recursive Descent Parser*. Diakses dari:

<https://www.geeksforgeeks.org/compiler-design/recursive-descent-parser/>